

✓ Practical No. 1 : Write a 'C' program to perform following Operations on Array: Create, Insert, Delete, Display

(I) Practical Significance

Writing a 'C' program to perform operations on an array helps in understanding fundamental data structures, memory management, and algorithm development. It provides a foundation for more complex programming tasks and real-world applications, reinforcing essential problem-solving skills in computer science.

(II) Industry/ Employer Expected Outcome

Writing a 'C' program to perform following Operations on Array: Create, Insert, Delete, Display will help Industry to assess the candidate's understanding of fundamental programming concepts, data structures, and problem-solving skills. By performing this experiment the student should

- (1) Properly initialize an array with a specified size.
- (2) Add elements at specified positions within the array while handling possible edge cases (e.g., inserting at the beginning, end, or beyond the current array size).
- (3) Remove elements from specified positions, ensuring the array's integrity and shifting elements accordingly.
- (4) Correctly print all the elements of the array in a readable format.

(III) Course Level Learning Outcomes

Perform basic operations on Arrays.

(IV) Laboratory Learning Outcome

Implement Array Operations.

(V) Relevant Affective domain related Outcome(s)

1. Follow safety practices.
2. Follow ethical practices.

(VI) Relevant Theoretical Background

In C programming, arrays are a fundamental data structure used to store a collection of elements of the same data type. Operations on arrays typically involve creating an array, inserting elements into the array, deleting elements from the array, and displaying the contents of the array. Here's a brief theoretical background for each of these operations:

1. Create(or Declare) Array

To create an array in C, you declare a variable of array type, specifying the data type of its elements and the number of elements it can hold.

For example: `int myArray[10];` // Creates an array of 10 integers

In C, arrays are zero-indexed, meaning the first element is at index 0 and the last element is at index (size - 1).

2. Insert into Array

Inserting elements into an array involves assigning values to specific elements within the array.

To insert an element at a specific index in the array, you can directly assign a value to that index. However, this will overwrite any existing value at that index.

If you want to insert an element and shift the existing elements to accommodate the new one, you would typically use a loop to move elements to make space for the new element.

3. Delete from Array

Deleting elements from an array involves removing elements from specific positions within the array.

To delete an element from an array, you typically shift the elements after the deleted element one position to the left to fill the gap created by the deletion.

You may also need to adjust the size of the array to reflect the removal of an element, which is not straightforward in C since arrays have fixed sizes once declared.

4. Display Array

Displaying the contents of an array involves iterating over the elements of the array and printing or displaying each element.

This can be done using a loop, such as a for loop, to iterate over the array and print each element.

For example:

```
for (int i = 0; i < size; i++) {  
    printf("%d ", myArray[i]);  
}
```

It's important to note that in C, arrays are fixed-size data structures, meaning their size cannot be changed once they are declared. To implement dynamic resizing or other more complex operations, you would typically need to use dynamic memory allocation techniques, such as `malloc()` and `free()`, along with pointer arithmetic. Additionally, error handling and boundary checks are crucial to ensure the correctness and robustness of array operations in C.

(VII) Algorithm

CAMPUSIFY

(VIII) Flow Chart

(IX) C Program Code



CAMPUSIFY

Algorithm:

Initialize :

- Declare ~~arr~~ [MAX - SIZE] and variables $n = 0$, choice, pos, and value

Menu loop:

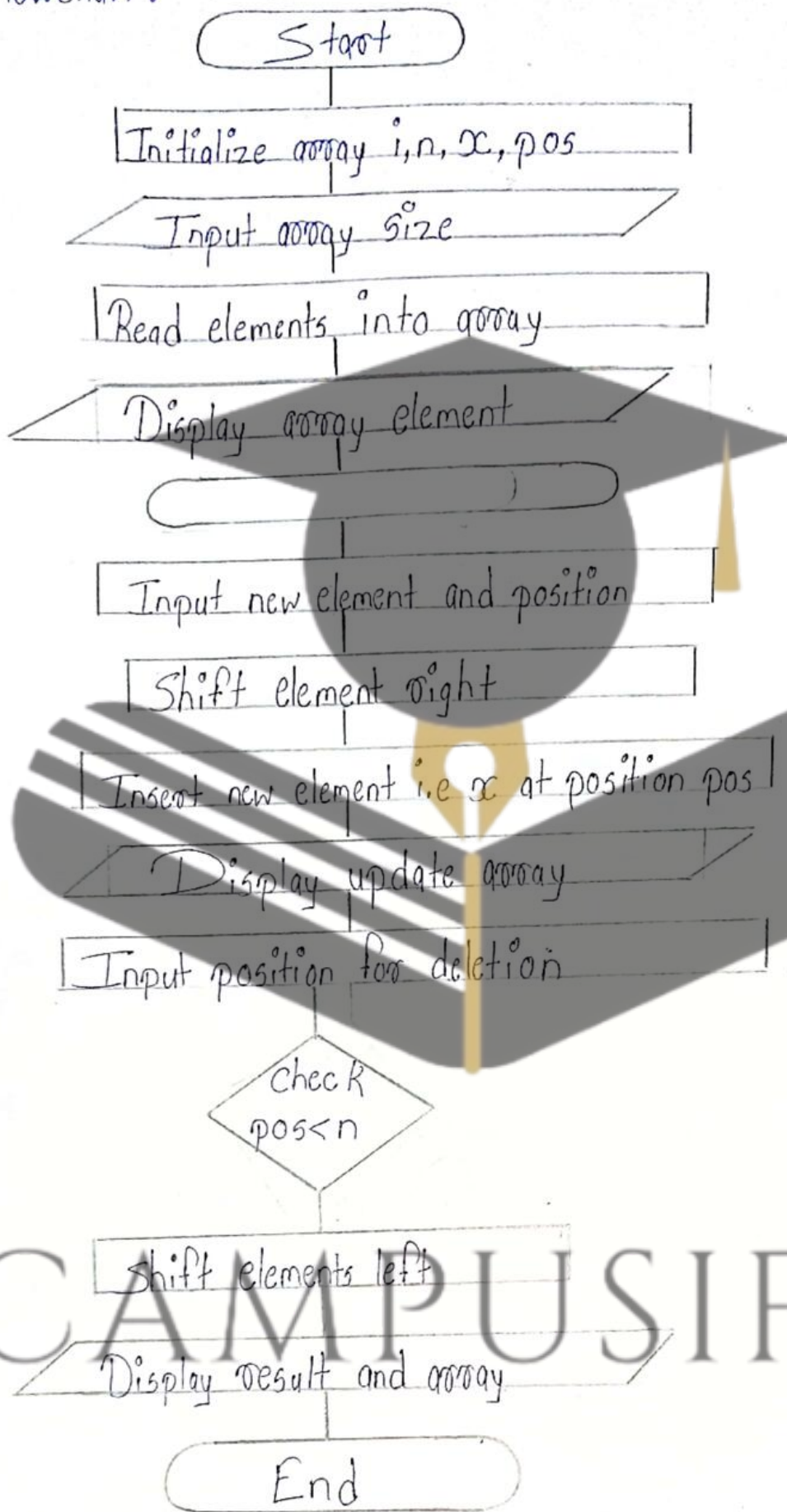
- Display menu options
- Read ~~choice~~ choice

Process Choic :

- Create Array :
 - Read n (number of elements)
 - If $n > \text{MAX_SIZE}$, set $n = \text{MAX_SIZE}$. If $n < 0$, set $n = 0$
 - Read n elements into ~~arr~~
- Insert Element :
 - Read pos (position) and value
 - Check if the array is full or pos is invalid ; print error if true
 - Shift elements to the right from pos
 - Insert value at pos
 - Increment n .
- Delete Element
 - Read pos (condition)
 - Check if n is 0 or pos is invalid, print error if true
 - Shift element to the left from pos
 - Decrement n
- Display Array
 - If n is 0, print "Array is empty"
 - Otherwise, print elements from ~~arr~~ [0] to ~~arr~~ [$n-1$]
- Exit
 - Print exit message and terminate loop.

Repeat ~~arr~~ until user chooses to exit

Flowchart:



C Program Code :

```
#include <stdio.h>
#define MAX_SIZE 100

void createArray (int arr[], int *n)
{
    printf("Enter the number of elements (up to %d):",
           MAX_SIZE);
    scanf("%d", n);
    if (*n > MAX_SIZE) {
        printf("Error: Number of elements exceed maximum
               allowed. size (%d).\n", MAX_SIZE);
        *n = MAX_SIZE; // Limit n to MAX_SIZE
    } else if (*n < 0) {
        printf("Error: Number of elements cannot be
               negative.\n");
        *n = 0; // set n to 0 if invalid input
        return;
    }
    printf("Enter the elements : \n");
    for (int i = 0; i < *n; i++) {
        scanf("%d", &arr[i]);
    }
}

void insertElement (int arr[], int *n, int pos, int value)
{
    if (*n >= MAX_SIZE) {
        printf("Error: Array is full, cannot insert new
               elements.\n");
        return;
    }
    if (pos < 0 || pos > *n) {
        printf("Invalid position\n");
        return;
    }
}
```

```

    for (int i = *n; i > pos; i--) {
        arr[i] = arr[i-1];
    }
    arr[pos] = value;
    (*n)++;
}

void deleteElement(int arr[], int *n, int pos)
{
    if (*n == 0) {
        printf("Error: Array is empty, cannot delete elements.\n");
        return;
    }
    if (pos < 0 || pos >= *n)
    {
        printf("Invalid position\n");
        return;
    }
    for (int i = pos; i < *n - 1; i++) {
        arr[i] = arr[i+1];
    }
    (*n)--;
}

void display Array(int arr[], int n)
{
    if (n == 0) {
        printf("Array is empty.\n");
        return;
    }
    printf("Array elements:");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

```

}

C Program Code : Continue -

```

int main()
{
    int arr [MAX - SIZE];
    int n = 0;    // Initialize n
    int choice, pos, value;
    do {
        printf("\nMenu: \n");
        printf("1. Create Array \n");
        printf("2. Insert Elements \n");
        printf("3. Delete Elements \n");
        printf("4. Display Array \n");
        printf("5. Exit \n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            Case 1 :
                Create Array (arr, &n);
                break;

            Case 2 :
                printf("Enter the position to insert (0 to %d): ", n);
                scanf("%d", &pos);
                printf("Enter the value to insert: ");
                scanf("%d", &value);
                insertElement(arr, &n, pos, value);
                break;

            Case 3 :
                printf("Enter the position to delete (0 to %d): ", n-1);
                scanf("%d", &pos);
                deleteElement(arr, &n, pos);
                break;
        }
    } while (choice != 5);
}

```

(X) Resources required

Sr. No.	Equipment Name with broad specification	Relevant LLO Number
1	Computer System with all necessary Peripherals and Internet Connectivity 'C' Compiler / GCC Compiler/ Online 'C' Compiler All	ALL

(XI) Precautions to be Followed

1. Ensure that all C statements must end with a semicolon (;).
2. Use white spaces in c to describe blanks and tabs.
3. Ensure use of Dynamic memory allocation to store Stack data
4. Follow safety practices.

(XII) Result (Output of Program) -

Menu :

1. Create Array
2. Insert Element
3. Delete Element
4. Display Array
5. Exit

Enter your choice : 1

Enter the number of elements (up to 100): 5

Enter the elements :

10 20 30 40 50

(XIII) Conclusion

We completed the practical - Write a 'C' program to perform the following Operations on Array: Create, Insert, Delete, Display, this practical is important in Data Structure Using C (313301) Manual.

(XIV) Practical Related Questions

1. Write a C program to find minimum and maximum element in an array
2. Write a C program to search a specific element in an array.

(XV) Exercise

1. List limitations of array in C.
2. Explain how sizeof operator works with arrays.

[Space to Write for Answers]

PRQ: Q1) Program -

```
#include <stdio.h>

void findMinMax(int arr[], int n, int *min, int *max) {
    *min = arr[0];
    *max = arr[0];

    for (int i = 1; i < n; i++) {
        if (arr[i] < *min) {
            *min = arr[i];
        }
    }

    int main() {
        int n;
        printf("Enter the number of elements in the array:");
        scanf("%d", &n);

        int arr[n];
        printf("Enter the elements of the array:\n");
        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
        }

        int min, max;
        findMinMax(arr, n, &min, &max);
    }
}
```

```

printf("Minimum elements: %d\n", min);
printf("Maximum elements: %d\n", max);

return 0;
}

```

(XVI) References / Suggestions for further Reading Software/Learning Websites

1. <https://www.javatpoint.com/data-structure-introduction>
2. <https://www.geeksforgeeks.org/introduction-to-data-structures/>

(XVII) Assessment Scheme

	Performance Indicators	Weightage %
	Process related (30 Marks)	60%
1.	Debugging ability	20%
2.	Correctness of Program codes	30%
3.	Quality of output achieved (LLO mapped)	10%
	Product related : 20 Marks	40%
4.	Completion and submission of practical in time	20%
5.	Answer to sample questions	20%
	Total (50 Marks)	100%

List of Student /Team Members

(1) (2)
 (3) (4)

Marks Obtained			Dated signature of Teacher
Process Related (30)	Product Related (20)	Total (50)	

P.R.Q :

Q2) Program:

#include <stdio.h>

int main() {

int n, i, searchElement, found = 0;

// Input the number of elements in the array

printf("Enter the number of elements in the array : ");

scanf("%d", &n);

int arr[n]; // Input elements of the array

printf("Enter %d elements : \n", n);

for (i = 0; i < n; i++) {

scanf("%d", &arr[i]);

}

// Input the element to search for

printf("Enter the element to search for : ");

scanf("%d", &searchElement);

// Linear search for the element

for (i = 0; i < n; i++) {

if (arr[i] == searchElement) {

printf("Element %d found at position %d.\n",
searchElement, i + 1);

found = 1;

break;

}

}

if (!found) {

printf("Element %d not found in the array.\n",
searchElement);

}

return 0;

}

✓ Practical No. 3 : Write a 'C' Program to Search a Particular Data from the Given Array of Strings using Linear Search Method

(I) Practical Significance

In C, a program using linear search can efficiently find a specific data within an array of Strings by sequentially checking each element until the target is found or the end of the array is reached. This method is straightforward and suitable for small or unsorted arrays.

(II) Industry/ Employer Expected Outcome

Demonstrate proficiency in writing C program. This includes understanding how to implement the algorithm correctly, handling edge cases, and writing clean, well-commented code. Additionally, students should be able to analyze the time complexity of their solution and possibly optimize it for better performance. Overall, industry seeks candidates who can effectively apply fundamental programming concepts to solve real-world problems. By performing this experiment student should-

1. Locate a specific data within an array of numbers, demonstrating the candidate's ability
2. to work with arrays, loops, and conditional statements
3. Write clean and well commented code
4. Use optimization technique for performance improvement

(III) Course Level Learning Outcomes

Perform basic operations on Arrays

(IV) Laboratory Learning Outcome

Implement Linear Search Method on Strings

(V) Relevant Affective domain related Outcome(s)

1. Follow safety practices.
2. Follow ethical practices.

(VI) Relevant Theoretical Background

The linear search is one of the most elementary and basic search methods. The specifics of the linear search will be covered in this blog post, along with its implementation in the C programming language, syntax examples, and samples of the expected result.

The fundamental search technique known as linear search that scans every element in a collection until the required element is discovered or the collection has been thoroughly searched. It is often referred to as sequential search. When dealing with short or unordered lists, this algorithm is especially helpful.

Here's how the linear search algorithm works:

- Start from the first element of the array.
- Compare the target value with the current element of the array.
- If the target value matches the current element, return the index of the element.
- If the target value does not match the current element, move to the next element in the array.
- Repeat steps 2-4 until the target value is found or until the end of the array is reached. If the target value is not found after checking all elements, return a special value (such as -1) to indicate that the element is not present in the array.

Linear search has a time complexity of $O(n)$, where n is the number of elements in the array. It is suitable for small arrays or when the elements are not sorted. However, for large arrays or when efficiency is critical, other search algorithms like binary search (which requires a sorted array) may be more appropriate.

(VII) Algorithm :

Step 1 : Start

Step 2 : Initialize the following :

- An array of strings `arr[]` with n elements (strings)
- A string variable `target` to hold the search string
- An integer variable `i` for looping

Step 3 : Input the target string to be searched

Step 4 : Loop through the array `arr[]` using a for loop from $i = 0$ to $n - 1$

- Step 4.1 : For each iteration, compare `arr[i]` with `target`
- Step 4.2 : If `arr[i]` matches `target`, go to Step 5

Step 5 : If a match is found :

- Print "Target string found at index i "
- Exit the loop

Step 6 : If no match is found after the loop ends :

- Print "Target string not found in the array."

Step 7 : End

CAMPUSIFY

(IX) C Program Code**(X) Resources required**

Sr. No.	Equipment Name with broad specification	Relevant Number LLO Number
1	Computer System with all necessary Peripherals and Internet Connectivity 'C' Compiler / GCC Compiler/ Online 'C' Compiler All	ALL

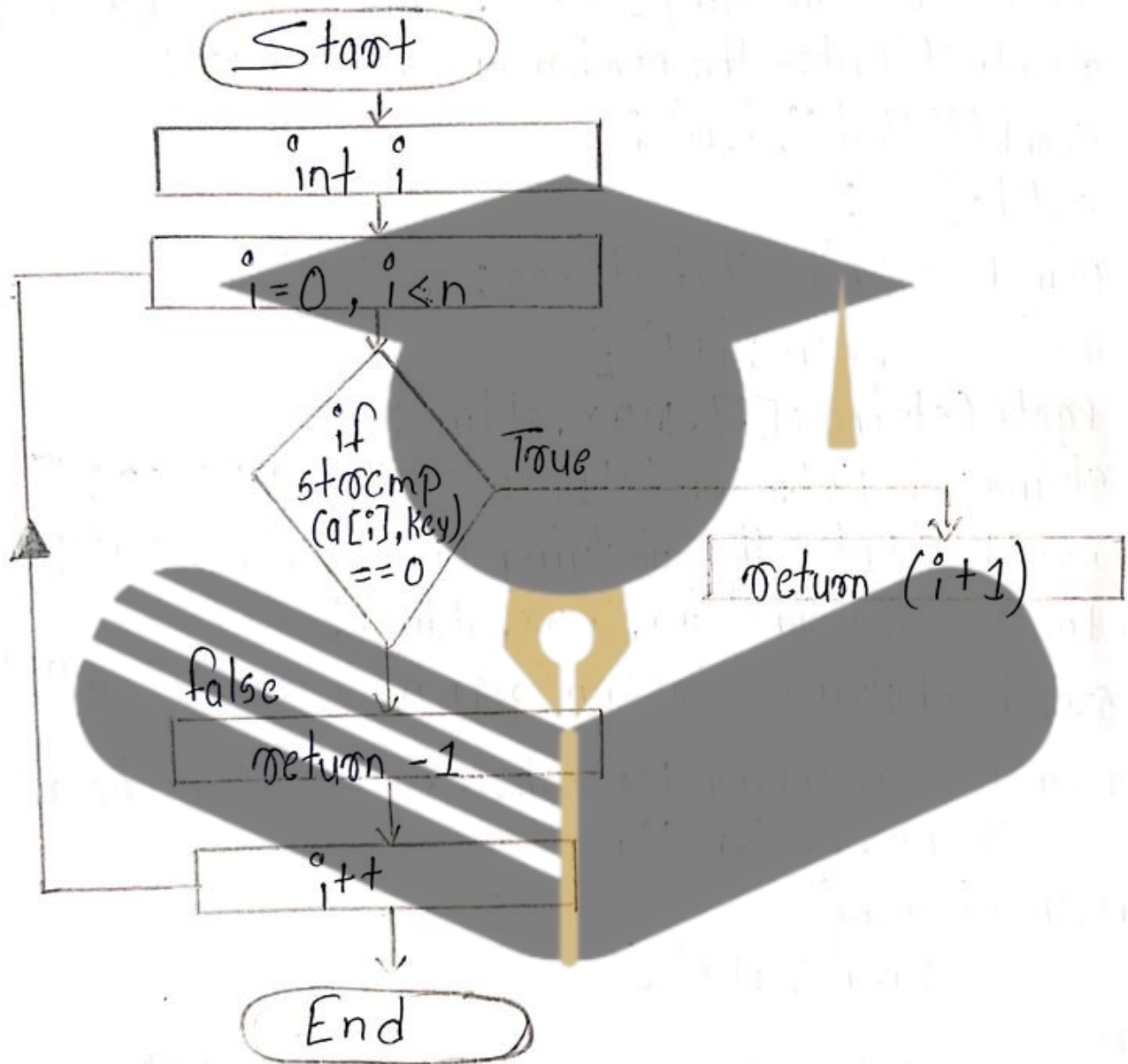
(XI) Precautions to be Followed

1. Ensure that all C statements must end with a semicolon (;).
2. Use white spaces in c to describe blanks and tabs.
3. Follow safety practices.

(XII) Result (Output of Program)

Enter the string to search : apple
Target string found at index 0

Flowchart :

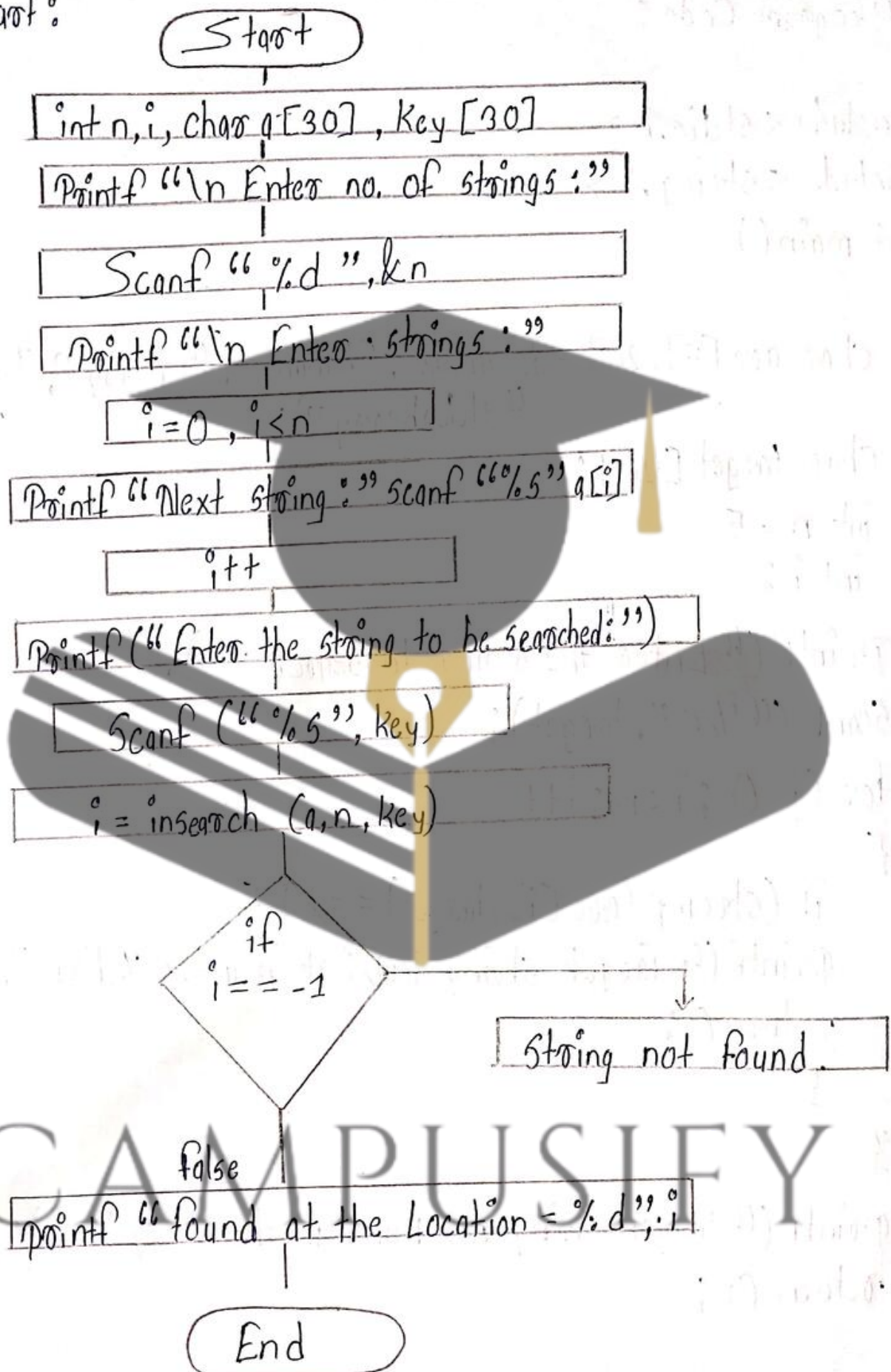


CAMPUSIFY

C Program Code :

```
#include <stdio.h>
#include <string.h>
int main()
{
    char arr[5][20] = {"apple", "banana", "cherry", "date",
                      "elderberry"};
    char target[20];
    int n = 5;
    int i;
    printf("Enter the string to search:");
    scanf("%s", target);
    for (i = 0; i < n; i++)
    {
        if (strcmp(arr[i], target) == 0) {
            printf("Target string found at index %d\n", i);
            return 0;
        }
    }
    printf("Target string not found in the array.\n");
    return 0;
}
```

Flowchart:



(XIII) Conclusion

~~We completed the practical write a 'C' program to perform the following operation on array: Create, Insert, Delete, Search.~~

Conclusion:

We successfully completed Data Structure Using C (313301) Practical No. 3 and Write a 'C' Program to Search a particular data from the given Array of Strings using Linear Search Method

(XIV) Practical Related Questions

1. Write a simple C program to perform linear search on an array of Strings to find Case-Insensitive search for a string
2. Write a simple C program to perform linear search on an array of Strings to find Checking for substring in each string

(XV) Exercise

1. Write applications of Linear Search Algorithm.
2. Modify the linear search function to perform a case-insensitive search.

[Space to Write for Answers]

PRQ-Q1) Program:

```
#include <stdio.h>
#include <string.h>
#define MAX 100

int main() {
    int n, i, found = 0;
    char searchString[MAX], strings[MAX][MAX];
    printf("Enter the number of strings: ");
    scanf("%d", &n);
    getchar();
    printf("Enter %d strings: \n", n);
    for (i = 0; i < n; i++) {
        fgets(strings[i], MAX, stdin);
    }
```

```

strings[i][strlen(strings[i], "\n")] = 0;
}
printf("Enter the string to search for: ");
fgets(searchString, MAX, stdin);
searchString[strlen(searchString, "\n")] = 0;
for (i = 0; i < n; i++) {
    if (strcmp(strings[i], searchString) == 0) {
        printf("String '%s' found at position %d.\n",
            searchString, i + 1);
        found = 1;
    }
}
if (!found) {
    printf("String '%s' not found in the array.\n", searchString);
    return 0;
}

```

(XVI) References / Suggestions for further Reading Software/Learning Websites

1. <https://www.javatpoint.com/data-structure-introduction>
2. <https://www.geeksforgeeks.org/introduction-to-data-structures/>

(XVII) Assessment Scheme

	Performance Indicators	Weightage %
	Process related (30 Marks)	60%
1.	Debugging ability	20%
2.	Correctness of Program codes	30%
3.	Quality of output achieved (LLO mapped)	10%
	Product related : 20 Marks	40%
4.	Completion and submission of practical in time	20%
5.	Answer to sample questions	20%
	Total (50 Marks)	100%

List of Student /Team Members

- (1) Pooja Pande (2) Shivam Jamnik
 (3) (4)

Marks Obtained			Dated signature of Teacher
Process Related (30)	Product Related (20)	Total (50)	

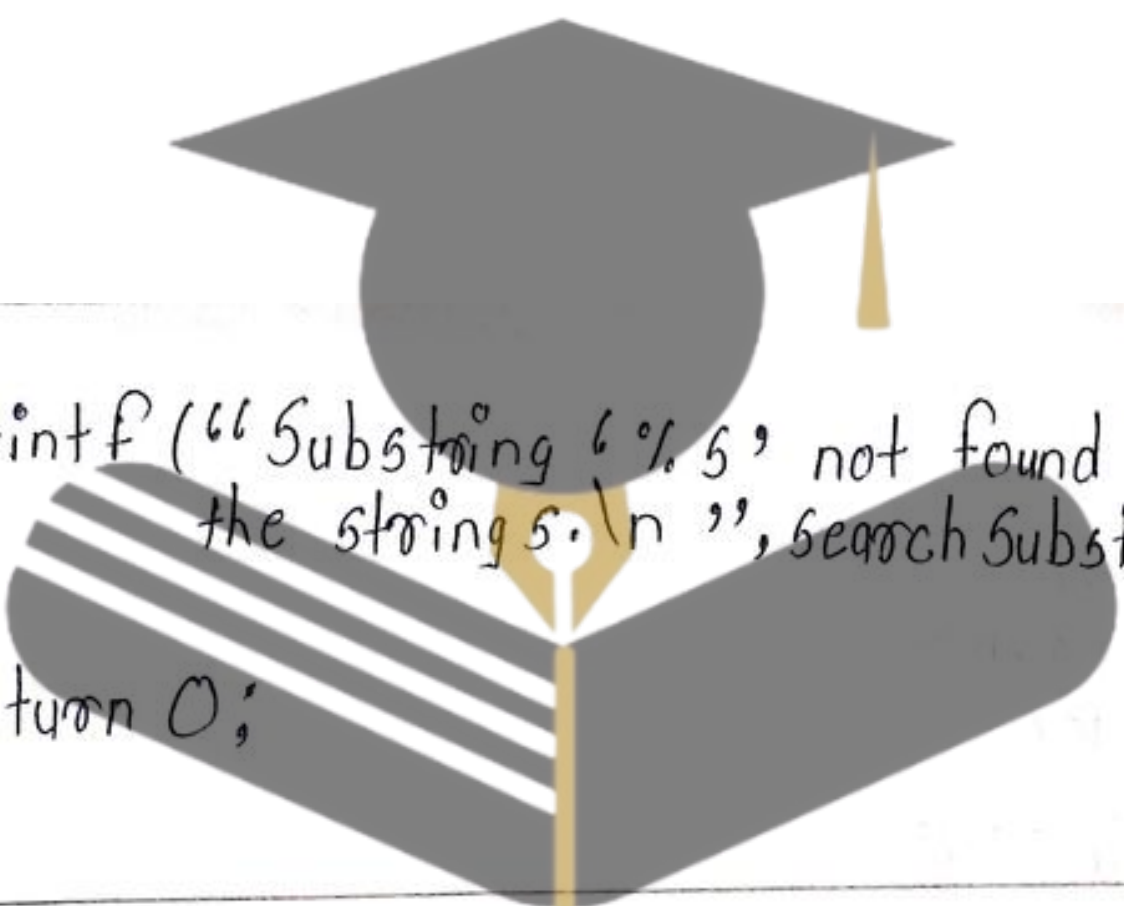
Program:

```

#include <stdio.h>
#include <string.h>
#define MAX 100

int main() {
    int n, i, found = 0;
    char searchSubstring[MAX], strings[MAX][MAX];
    printf("Enter the number of strings: ");
    scanf("%d", &n);
    getchar();
    printf("Enter %d strings:\n", n);
    for (i = 0; i < n; i++) {
        fgets(strings[i], MAX, stdin);
        strings[i][strlen(strings[i], "\n")] = 0;
        printf("Enter the substring to search for: ");
        fgets(searchSubstring, MAX, stdin);
        searchSubstring[strlen(searchSubstring, "\n")] = 0;
        printf("Searching for substring '%s' in the array of\n",
            strings,
            searchSubstring);
        for (i = 0; i < n; i++) {
            if (
                strstr(strings[i], searchSubstring) != NULL) {
                printf("Substring '%s' found in string '%s' at position\n",
                    "%d.\n", searchSubstring, strings[i], i + 1);
                found = 1;
            }
        }
        if (!found)

```



```
{  
printf("Substring '%s' not found in any of  
the strings.\n", searchSubstring);  
}  
return 0;
```

```
}
```

CAMPUSIFY

✓ Practical No. 4 : Write a 'C' program to Search a Particular data from the Given Array of Numbers using Binary Search Method

(I) Practical Significance

Binary search is a highly efficient algorithm for finding a specific value in a sorted array. It significantly reduces the search time complexity to $O(\log n)$ by repeatedly dividing the search interval in half, making it particularly useful for large datasets.

(II) Industry/Employer Expected Outcome

The binary search algorithm's efficiency is crucial in various industries, particularly in database management, information retrieval, and software development.

1. Enable rapid data lookup and retrieval
2. Enhance performance in applications like search engines, data analytics platforms, and inventory management systems
3. Lead to faster query responses and improved user experiences.

(III) Course Level Learning Outcomes

Perform operations on Stack using Array and Linked List Implementations.

(IV) Laboratory Learning Outcome

Implement Binary search method on numbers.

(V) Relevant Affective Domain Outcome(s)

1. Follow safety practices.
2. Follow ethical practices.

(VI) Relevant Theoretical Background

Binary Search is an interval searching algorithm used to search for an item in the sorted list. It works by repeatedly dividing the list into two equal parts and then searching for the item that is the part where it can possibly exist.

Unlike linear search, there are a few conditions for applying binary search: The list must be sorted. Random access to the list member.

Explanation

Initialize Pointers : left is the starting index, and right is the ending index of the array. Loop Until the Pointers Meet : Continue the loop while left is less than or equal to right.

Calculate Middle Index: Find the middle index mid of the current search range using $(\text{left} + \text{right}) / 2$.

2. To avoid potential overflow, it's better to use $\text{left} + (\text{right} - \text{left}) / 2$. Compare Middle Element with Target:

If $\text{arr}[\text{mid}]$ equals the target, return mid (index of the target).

If $\text{arr}[\text{mid}]$ is less than the target, narrow the search to the right half ($\text{left} = \text{mid} + 1$).

If $\text{arr}[\text{mid}]$ is greater than the target, narrow the search to the left half ($\text{right} = \text{mid} - 1$).

Return Result: If the loop ends without finding the target, return -1 indicating the target is not in the array.

(VII) Algorithm :

Input: A sorted array arr of n elements and a target value to search for

Output: The index of the target in the array if found; otherwise, -1.

1) Initialize Pointers:

- Set left to 0 (the starting index of the array).
- Set right to $n-1$ (the ending index of the array).

2) While loop:

- Repeat while left is less than or equal to right:

1) Calculate Middle:

- Compute the middle index: $mid = left + (right - left) / 2$

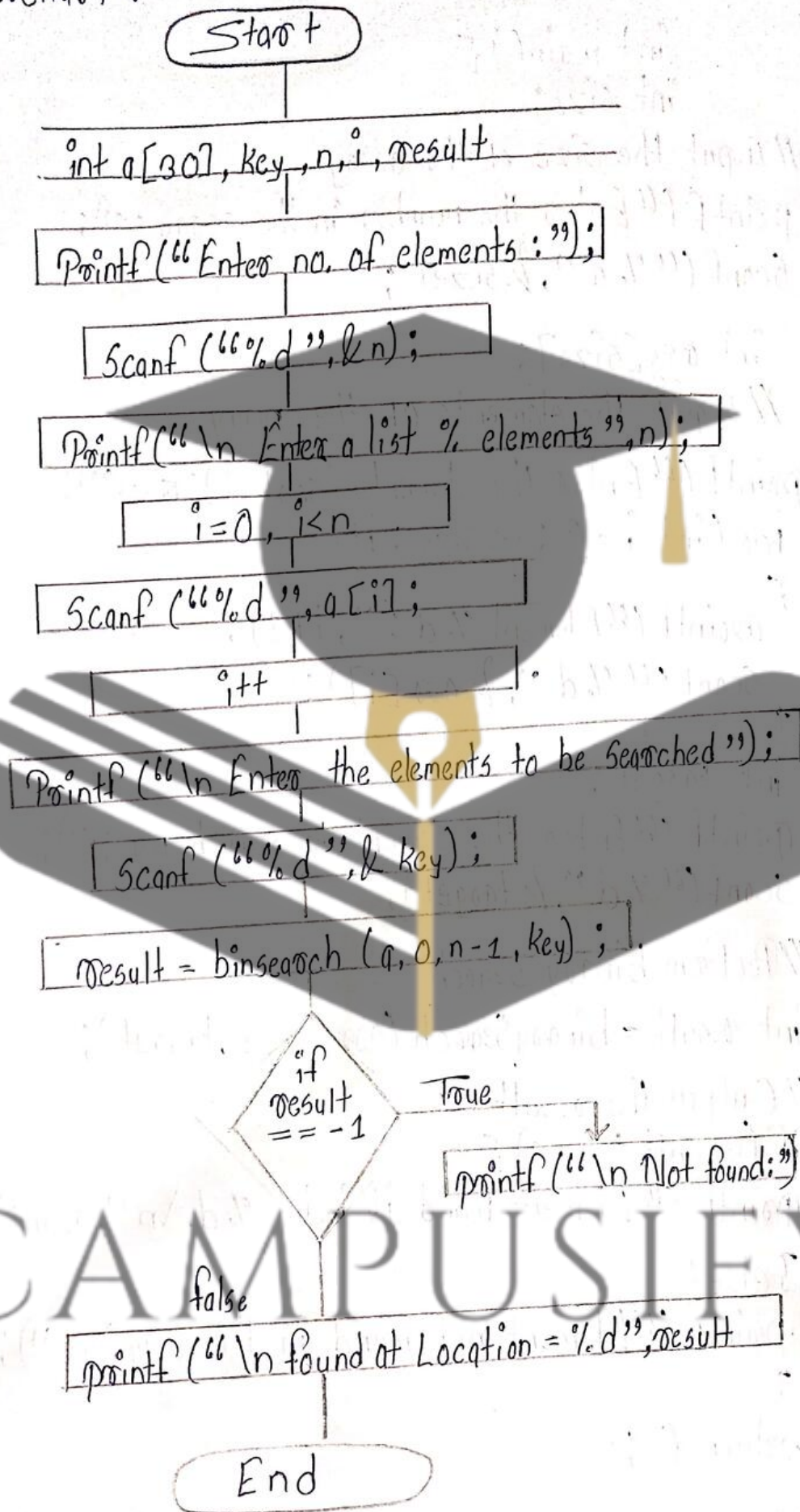
2) Compare with Target:

- If $arr[mid]$ is equal to the target:
 - Return mid (index of the target)
- If $arr[mid]$ is less than the target:
 - Set left to $mid + 1$ (Ignore the left half)
- If $arr[mid]$ is greater than the target:
 - Set right to $mid - 1$ (Ignore the right half)

3) Return Result:

- If the target is not found, return -1.

Flowchart :



(IX) C Program Code :

```
#include <stdio.h>

int binarySearch(int arr[], int size, int target)
{
    int left = 0;
    int right = size size - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) {
            return mid; // Target found
        }
        if (arr[mid] < target) { // If target is greater, Ignore left half
            left = mid + 1;
        }
        else { // If target is smaller, ignore right half
            right = mid - 1;
        }
    }
    return -1; // Target not found
}
```

C Program Code : Continue -

```
int main() {  
    int size;  
    // Input the size of the array  
    printf("Enter the number in the array :");  
    scanf("%d", &size);  
    int arr[size];  
    // Input the elements of the array  
    printf("Enter the elements (sorted): \n");  
    for (int i = 0; i < size; i++)  
    {  
        printf("Element %d : ", i + 1);  
        scanf("%d", &arr[i]);  
    }  
    int target;  
    printf("Enter the element to search for :");  
    scanf("%d", &target);  
    // Perform binary Search  
    int result = binarySearch(arr, size, target);  
    // Output the result  
    if (result != -1) {  
        printf("Element found at index %d. \n", result);  
    }  
    else {  
        printf("Element not found in the array. \n");  
    }  
    return 0;  
}
```

}

(X) Resources required		Relevant LLO Number
Sr. No.	Equipment Name with broad specification	
1	Computer System with all necessary Peripherals and Internet Connectivity 'C' Compiler / GCC Compiler/ Online 'C' Compiler All	ALL

(XI) Precautions to be Followed

1. Ensure that all C statements must end with a semicolon (;).
2. Use white spaces in c to describe blanks and tabs.
3. Ensure use of Dynamic memory allocation to store Stack data
4. Follow safety practices.

(XII) Result (Output of Program)

Enter the number of elements in the array : 1
 Enter the elements (sorted) :
 Element 1 : 1
 Enter the element to search for : 1
 Element found at index 0.

(XIII) Conclusion

We completed DSU (313301) Practical No-4 and -
 Write a 'C' program to search a particular data from the
 given array of numbers using Binary Search Method.

(XIV) Practical Related Questions

1. Write a program to find the first and last occurrence of the element 3 in an array of 20 integers using binary search.
2. Given an array of 15 integers, write a program to find the two middle elements using binary search.

(XV) Exercise

1. State the applications of Binary Search
2. Compare Binary Search with Linear Search
3. Discuss and write the requirements for using binary search on an array of numbers.

[Space to Write for Answers]

PRO - Q1)

Program:

```
#include <stdio.h>
```

```
int findFirstOccurrence(int arr[], int n, int target) {
```

```
    int low = 0, high = n - 1, result = -1;
```

```
    while (low <= high) {
```

```
        int mid = low + (high - low) / 2;
```

```
        if (arr[mid] == target) {
```

```
            result = mid;
```

```
            high = mid - 1;
```

```
        } else if (arr[mid] > target) {
```

```
            high = mid - 1;
```

```
        } else {
```

```
            low = mid + 1;
```

```
        }
```

```
    }
```

```
    return result;
```

```
}
```

```
int findLastOccurrence(int arr[], int n, int target) {
```

```
    int low = 0, high = n - 1, result = -1;
```

```
    while (low <= high) {
```

```
        int mid = low + (high - low) / 2;
```

```
        if (arr[mid] == target) {
```

```
            result = mid;
```

```
            low = mid + 1;
```

```

    } else if (arr[mid] > target)
    {
        high = mid - 1;
    } else {
        low = mid + 1;
    }
}

```

```

return result;
}

```

```

int main() {
    int arr[20], i, target = 3;
    printf("Enter 20 sorted integers: \n");
    for (i = 0; i < 20; i++)

```

(XVI) References / Suggestions for further Reading Software/Learning Websites

1. <https://www.javatpoint.com/data-structure-introduction>
2. <https://www.geeksforgeeks.org/introduction-to-data-structures/>

(XVII) Assessment Scheme

	Performance Indicators	Weightage %
	Process related (30 Marks)	60%
1.	Debugging ability	20%
2.	Correctness of Program codes	30%
3.	Quality of output achieved (LLO mapped)	10%
	Product related : 20 Marks	40%
4.	Completion and submission of practical in time	20%
5.	Answer to sample questions	20%
	Total (50 Marks)	100%

List of Student /Team Members(1) Prath Pande(2) Shivam Jamnik

(3)

(4)

Marks Obtained			Dated signature of Teacher
Process Related (30)	Product Related (20)	Total (50)	

```
{
    scanf ("%d", &arr[i]);
}
int first = findFirstOccurrence (arr, 20, target);
int last = findLastOccurrence (arr, 20, target);
if (first != -1 && last != -1)
{
    printf ("The first occurrence of %d is at position\n", target, first + 1);
    printf ("The last occurrence of %d is at position\n", target, last + 1);
} else {
    printf ("Elements %d not found in the array.\n", target);
}
return 0;
}
```

CAMPUSIFY

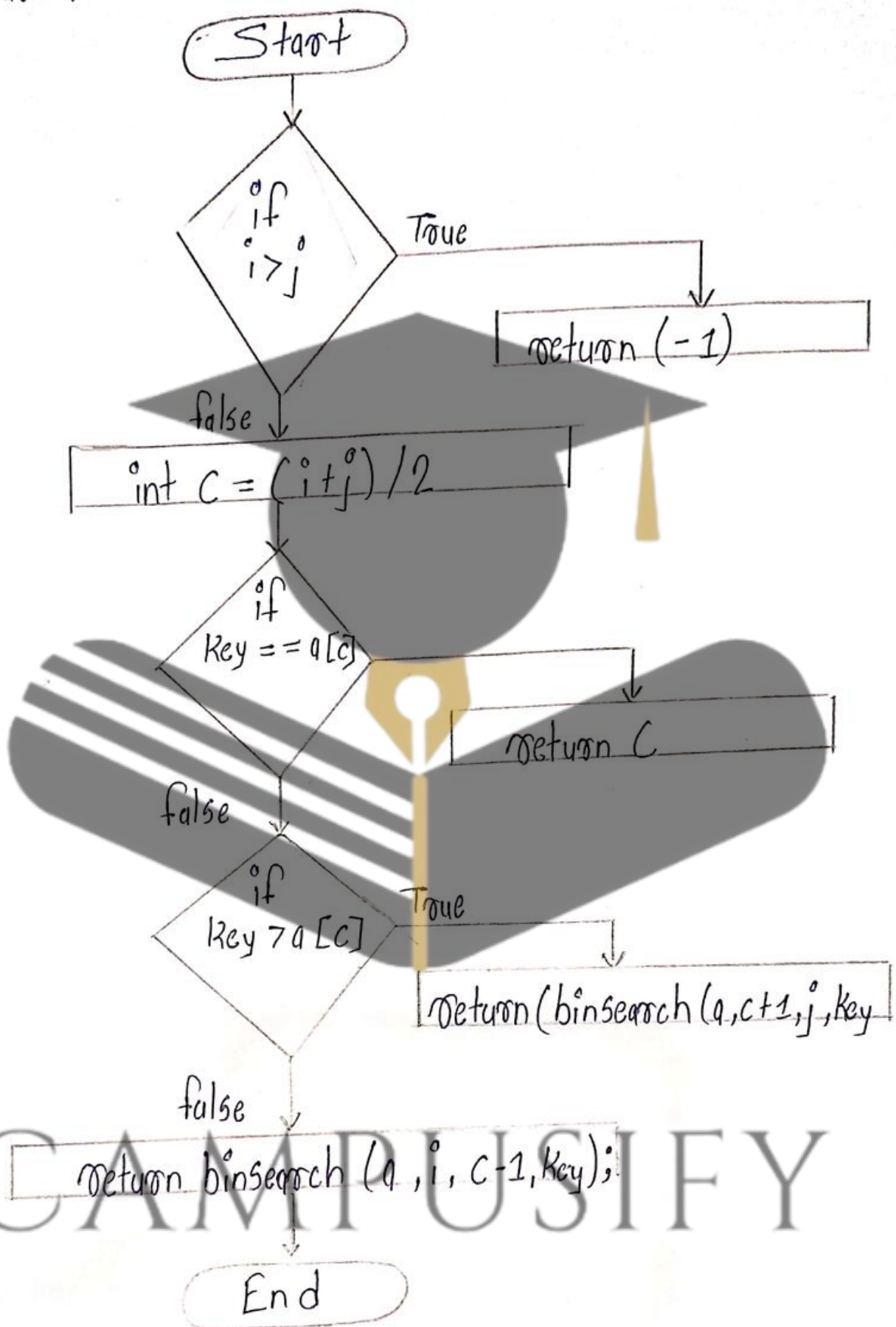
PRQ:

Q2) Program:

```
#include <stdio.h>
int main() {
    int arr[15], i;
    int mid1, mid2;
    // Input the array of 15 integers
    printf("Enter 15 integers: \n");
    for (i = 0; i < 15; i++)
    {
        scanf("%d", &arr[i]);
    }
    // calculate the indices of the two middle elements
    mid1 = (15 - 1) / 2; // middle index
    mid2 = mid1 + 1; // second middle index
    // Print the two middle elements
    printf("The two middle elements are: %d and %d \n",
        arr[mid1], arr[mid2]);
    return 0;
}
```

CAMPUSIFY

Flowchart:



✓ Practical No. 6 : Write a 'C' Program to Sort an Array of Numbers using Bubble Sort Method

(I) Practical Significance

Bubble sort is one of the simplest sorting algorithms that works by repeatedly stepping through the list to be sorted, comparing each pair of adjacent items, and swapping them if they are in the wrong order. This process is repeated until no swaps are needed, which indicates that the list is sorted.

(II) Industry/ Employer Expected Outcome

Through this practical student should

1. Illustrate basic algorithmic principles, such as iteration, comparison, and swapping.
2. Use Bubble sort for small datasets
3. Highlight importance of algorithmic efficiency
4. Understand Timecomplexity

(III) Course Level Learning Outcomes

Apply different Searching and Sorting methods.

(IV) Laboratory Learning Outcome

Apply Bubble Sort method for Sorting Numbers

(V) Relevant Affective domain related Outcome(s)

1. Follow safety practices.
2. Follow ethical practices.

(VI) Relevant Theoretical Background

Bubble sort is one of the simplest sorting algorithms that works by repeatedly stepping through the list to be sorted, comparing each pair of adjacent items, and swapping them if they are in the wrong order. This process is repeated until no swaps are needed, which indicates that the list is sorted.

Key Characteristics:

Simple Implementation: Easy to understand and implement.

Comparison-Based: Each element is compared with its adjacent element. In-Place Sorting: Requires a constant amount of additional memory space.

Stable Sort: Maintains the relative order of records with equal keys (i.e., values).

In summary, the bubble sort algorithm repeatedly swaps adjacent elements until the array is sorted. The algorithm has a time complexity of $O(n^2)$, which makes it inefficient for large data sets.

Working of Bubble Sort

Suppose we are trying to sort the elements in **ascending order**.

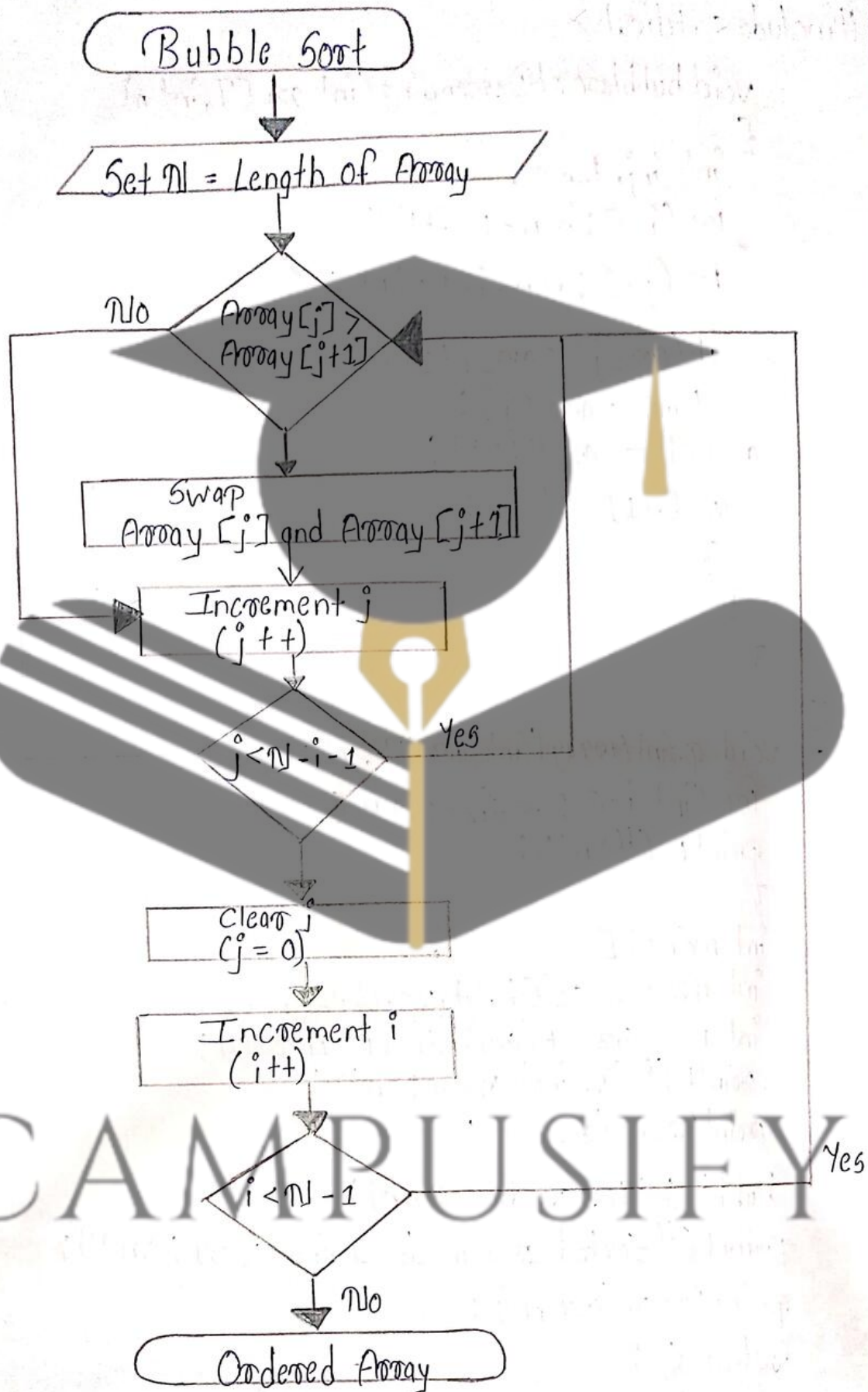
1. First Iteration (Compare and Swap)

Starting from the first index, compare the first and the second elements. If the first element is greater than the second element, they are swapped.

Now, compare the second and the third elements. Swap them if they are not in order. The above process goes on until the last element.



Flowchart :



(VII) Algorithm :

Step 1 : Start Bubble Sort

Step 2 : input a List

Step 3 : sorted = False

Step 5 : integer swaps = 0

Step 6 : for $I = 1$ to list.Count - 1

Step 7 : if list[i-1] > list[i] go to step number 8 else to step 6

Initialize temp = list[i-1]

list[i-1] = list[i]

list[i] = temp

Step 8 : swaps = swaps + 1

Step 9 : If swaps == 0 go to step 15 else go to step 16

~~Step 10 : Update variable sorted~~

Step 10 : Update variable sorted with Boolean value "True"

Step 11 : Variable Sorted = False

Step 12 : Output 'List'

Step 13 : End Bubble Sort

CAMPUSIFY

(IX) C Program Code

```
#include <stdio.h>
int main ()
{
    int n, i, j, swap;
    printf ("Enter number of elements \n");
    scanf ("%d", &n);
    int array [n];
    printf ("Enter %d integers \n", n);
    for (i = 0; i < n; i++)
    {
        scanf ("%d", &array[i]);
    }
    for (j = 0; j < n - 1; j++)
    {
        if (array[j] > array[j+1])
        {
            swap = array[j];
            array[j] = array[j+1];
            array[j+1] = swap;
        }
    }
    printf ("Sorted list in ascending order: \n");
    for (i = 0; i < n; i++)
        printf ("%d \n", array[i]);
    return 0;
}
```

(X) Resources required

Sr. No.	Equipment Name with broad specification	Relevant LLO Number
1	Computer System with all necessary Peripherals and Internet Connectivity 'C' Compiler / GCC Compiler/ Online 'C' Compiler All	ALL

(XI) Precautions to be Followed

1. Ensure that all C statements must end with a semicolon (;).
2. Use white spaces in c to describe blanks and tabs.
3. Ensure use of Static memory allocation using fixed memory size
4. Follow safety practices.

(XII) Result (Output of Program)

Unsorted array :

64 34 25 12 22 11 90

Sorted array :

11 12 22 25 34 64 90

(XIII) Conclusion

We Successfully completed DSU (313301) Practical No.6 and Wrote a 'C' program to Sort an Array of numbers using Bubble Sort Method.

CAMPUSIFY

(XIV) Practical Related Questions

- (1) Optimize the Bubble Sort algorithm to stop early if the array is already sorted.
- (2) Modify the Bubble Sort algorithm to sort an array in descending order.

(XV) Exercise

- (1) Enlist Applications of Bubble Sort.
- (2) Explain why Bubble sort is called as stable Sort.

[Space to Write for Answers]

PRQ - Q1) Program:

```
#include <stdio.h>
```

```
void bubbleSort (int arr [], int n)
```

```
{
```

```
    int i, j;
```

```
    int swapped;
```

```
    for (i = 0; i < n - 1; i++)
```

```
    {
```

```
        swapped = 0;
```

```
        for (j = 0; j < n - 1; j++)
```

```
        {
```

```
            if (arr[j] > arr[j+1])
```

```
            {
```

```
                int temp = arr[j];
```

```
                arr[j] = arr[j+1];
```

```
                arr[j+1] = temp;
```

```
                swapped = 1;
```

```
            }
```

```
        }
```

```
        if (swapped == 0)
```

```
        {
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
void printArray (int arr [], int size) {
```

```
for (int i=0; i < size; i++)  
{  
    printf("%d ", arr[i]);  
}  
printf("\n");  
}  
int main()  
{  
    int arr[10] = {64, 34, 25, 12, 22, 11,  
                   90, 78, 56, 45};  
    int n = sizeof(arr)/sizeof(arr[0]);  
    printf("Unsorted array: \n");  
    printArray(arr, n);  
    bubbleSort(arr, n);  
    printf("Sorted array: \n");  
    printArray(arr, n);  
    return 0;  
}
```

Output :

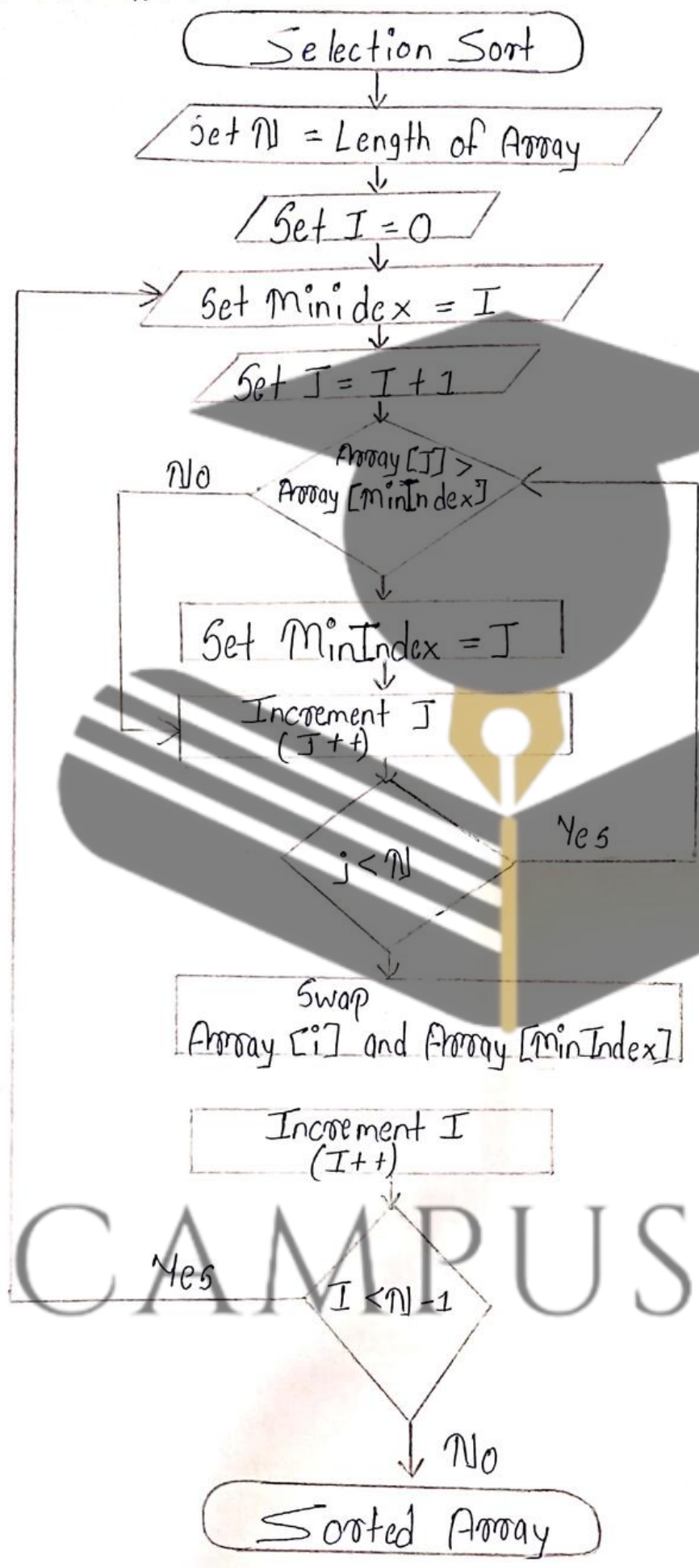
Unsorted array:

64 34 25 12 22 11 90 78 56 45

Sorted array:

11 12 22 25 34 45 56 64 78 90

FlowChart :



(X) Resources required

Sr. No.	Equipment Name with broad specification	Relevant LLO Number
1	Computer System with all necessary Peripherals and Internet Connectivity 'C' Compiler / GCC Compiler/ Online 'C' Compiler All	ALL

(XI) Precautions to be Followed

1. Ensure that all C statements must end with a semicolon (;).
2. Use white spaces in c to describe blanks and tabs.
3. Ensure use of Dynamic memory allocation to store Stack data
4. Follow safety practices.

(XII) Result (Output of Program)**(XIII) Conclusion**

We Successfully completed Data Structure Using C (313301) Practical No. 8 and wrote a 'C' program to sort an Array of numbers using the Selection Sort Method.

CAMPUSIFY

(XIV) Practical Related Questions

1. Modify the Selection Sort algorithm to handle arrays containing negative numbers.
2. Adapt the Selection Sort algorithm to sort an array of floating-point numbers.

(XV) Exercise

1. Write Applications of Selection Sort.
2. Compare Selection Sort with Bubble Sort.

[Space to Write for Answers]

PRO: Q1) Program:

```
#include <stdio.h>
void selectionSort(int arr[], int n) {
    int i, j, min_idx;
    for (i = 0; i < n - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        int temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }
}
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
int main() {
    int arr[] = {64, -25, 12, 22, -11, 0, 45, -9};
```

```

    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array: \n");
    printArray(arr, n);
    selectionSort(arr, n);
    printf("\n Sorted array: \n");
    printArray(arr, n);
    return 0;
}

```

(XVI) References / Suggestions for further Reading Software/Learning Websites

1. Selection Sort – Data Structure and Algorithm Tutorials - GeeksforGeeks
2. Selection Sort - javatpoint
3. C Program for Selection Sort? (tutorialspoint.com)

(XVII) Assessment Scheme

	Performance Indicators	Weightage %
	Process related (30 Marks)	60%
1.	Debugging ability	20%
2.	Correctness of Program codes	30%
3.	Quality of output achieved (LLO mapped)	10%
	Product related : 20 Marks	40%
4.	Completion and submission of practical in time	20%
5.	Answer to sample questions	20%
	Total (50 Marks)	100%

List of Student /Team Members

- (1) Parth Pande (2) Shivam Jambik
 (3) (4)

Marks Obtained			Dated signature of Teacher
Process Related (30)	Product Related (20)	Total (50)	

PRQ : Q2) Program -

```
#include <stdio.h>
```

```
void swap(float *xp, float *yp)
```

```
{
```

```
    float temp = *xp;
```

```
    *xp = *yp;
```

```
    *yp = temp;
```

```
}
```

```
void selectionSort(float arr[], int n)
```

```
{
```

```
    int i, j, min_idx;
```

```
    for (i = 0; i < n - 1; i++)
```

```
    {
```

```
        min_idx = i;
```

```
        for (j = i + 1; j < n; j++)
```

```
        {
```

```
            if (arr[j] < arr[min_idx])
```

```
            {
```

```
                min_idx = j;
```

```
            }
```

```
        }
```

```
        swap(&arr[min_idx], &arr[i]);
```

```
    }
```

```
}
```

```
void printArray(float arr[], int size)
```

```
{
```

```
    int i;
```

```
    for (i = 0; i < size; i++)
```

```
    {
```

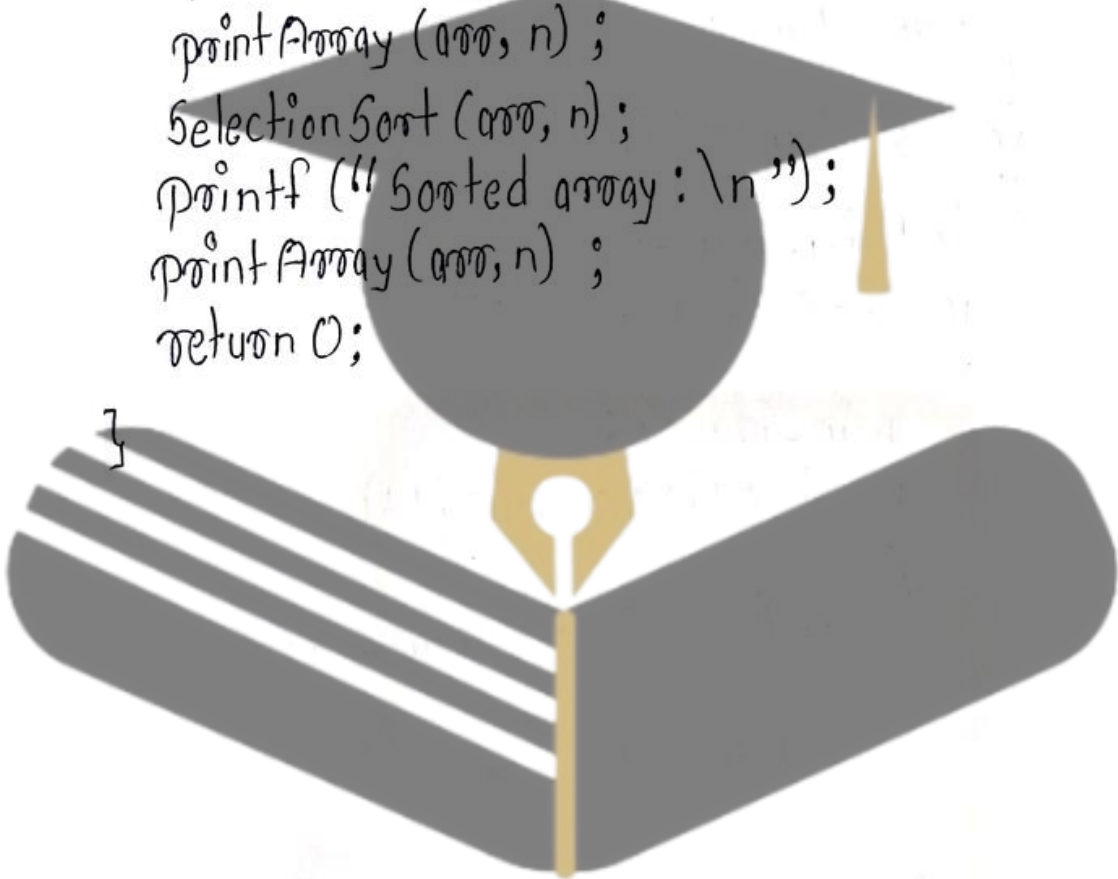
```
        printf("%.2f", arr[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int main ()  
{  
    float arr[] = { 64.5, 34.2, 25.3, 12.1, 22.9,  
                    11.4, 90.0 };  
    int n = sizeof(arr) / sizeof(arr[0]);  
    printf (" Original array : \n ");  
    printArray (arr, n) ;  
    selectionSort (arr, n) ;  
    printf (" Sorted array : \n ");  
    printArray (arr, n) ;  
    return 0;  
}
```



CAMPUSIFY

✓ Practical No. 10 : Write a 'C' Program to Sort an Array of Numbers using Insertion Sort Method

(I) Practical Significance

Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list. It is a **stable sorting** algorithm, meaning that elements with equal values maintain their relative order in the sorted output.

(II) Industry/ Employer Expected Outcome

Through this practical student should

1. Sort Relatively small data
2. Work on incremental data changes
3. Maintain sorted Lists
4. Use Insertion sort for small partitions of data

(III) Course Level Learning Outcomes

Apply different searching and sorting methods

(IV) Laboratory Learning Outcome

Apply Insertion Sort for Sorting Numbers

(V) Relevant Affective domain related Outcome(s)

1. Follow safety practices.
2. Follow ethical practices.

(VI) Relevant Theoretical Background

Insertion Sort is a simple sorting algorithm that builds the final sorted array one item at a time. It is much less efficient on large lists than more advanced algorithms. However, it has the advantage of being easy to implement and efficient for small data sets.

Insertion sort works similar to the sorting of playing cards in hands. It is assumed that the first card is already sorted in the card game, and then we select an unsorted card. If the selected unsorted card is greater than the first card, it will be placed at the right side; otherwise, it will be placed at the left side. Similarly, all unsorted cards are taken and put in their exact place.

The same approach is applied in insertion sort. The idea behind the insertion sort is that first take one element, iterate it through the sorted array. Although it is simple to use, it is not appropriate for large data sets as the time complexity of insertion sort in the average case and worst case is $O(n^2)$, where n is the number of items. Insertion sort is less efficient than the other sorting algorithms like heap sort, quick sort, merge sort, etc.

Now that we have clarity of the concept and working of the Insertion Sort program in C, let us take an example array and dry run with illustrations to understand the working in a step by step manner:-

Actual Array

0	1	2	3	4
6	2	1	3	5

(VII) Algorithm

The simple steps of achieving the insertion sort are listed as follows -

- Step 1: If the element is the first element, assume that it is already sorted. Return 1
- Step 2: Pick the next element, and store it separately in a key
- Step 3: Now, compare the key with all elements in the sorted array
- Step 4: If the element in the sorted array is smaller than the current element, then element. Else, shift greater elements in the array towards the right.
- Step 5: Insert the value.
- Step 6: Repeat until the array is sorted.

(VIII) Flow Chart

CAMPUSIFY

(IX) C Program Code

```

#include <stdio.h>
int main (void)
{
    int n, i, j, temp;
    int arr[64];
    printf("Enter number of elements \n");
    scanf("%d", &n);
    printf("Enter %d integers \n", n);
    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    for (i = 1; i < n; i++)
    {
        j = 1;
        while (j > 0 && arr[j-1] > arr[j])
        {
            temp = arr[j];
            arr[j] = arr[j-1];
            arr[j-1] = temp;
            j--;
        }
    }
    printf("Sorted list in ascending order : \n");
    for (i = 0; i < n; i++)
    {
        printf("%d \n", arr[i]);
    }
    return 0;
}

```

(X) Resources required		Relevant LLO Number
Sr. No.	Equipment Name with broad specification	
1	Computer System with all necessary Peripherals and Internet Connectivity 'C' Compiler / GCC Compiler/ Online 'C' Compiler All	ALL

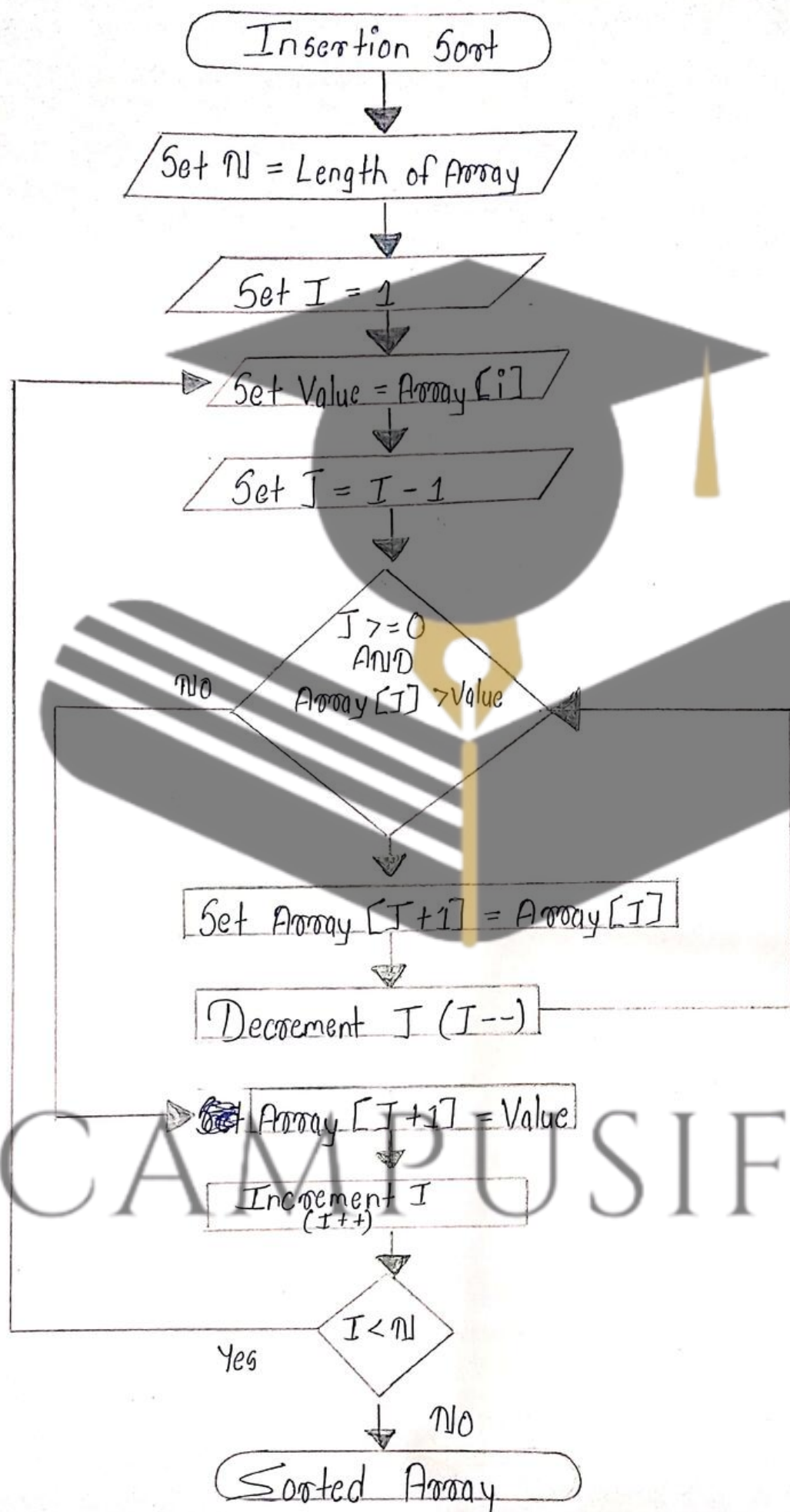
(XI) Precautions to be Followed

1. Ensure that all C statements must end with a semicolon (;).
2. Use white spaces in c to describe blanks and tabs.
3. Ensure use of Dynamic memory allocation to store Stack data
4. Follow safety practices.

(XII) Result (Output of Program)**(XIII) Conclusion**

We successfully completed Data Structure Using C (313301) Practical No. 10 and Write a 'C' program to sort an Array of Numbers using Insertion Sort Method

Flowchart :



(XIV) Practical Related Questions

1. Modify the Insertion Sort algorithm to use binary search for finding the correct position to insert the current element. Implement this modified algorithm and compare its performance with the standard Insertion Sort.
2. Use the Insertion Sort algorithm to count the number of inversions in an array. An inversion is a pair of elements where the earlier element is greater than the later element.

(XV) Exercise

1. Write Time and Space Complexities of Insertion Sort.
2. List Applications of Insertion Sort
3. How does insertion sort handle duplicate values in the array?

[Space to Write for Answers]

P.R.Q : Q2) Program

```

#include <stdio.h>
// Function to perform Insertion Sort and count the number of inversions
int insertionSortAndCount (int arr [], int n)
{
    int inversionCount = 0;
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        // Shift elements of arr [0...i-1], that are greater than key,
        // to one position ahead of their current position
        while (j >= 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j = j - 1;
            inversionCount++; // Count each inversion
        }
        arr[j+1] = key;
    }
    return inversionCount;
}

// Function to print an array
void printArray (int arr [], int n) {

```

```

for (int i = 0; i < n; i++) {
    printf("%d", arr[i]);
}
printf("\n");
}
// Main function
int main()
{
    int arr[] = {8, 4, 2, 1};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array: \n");
    printArray(arr, n);
}

```

(XVI) References / Suggestions for further Reading Software/Learning Websites

1. Insertion Sort - Data Structure and Algorithm Tutorials - GeeksforGeeks
2. Insertion Sort - javatpoint

(XVII) Assessment Scheme

	Performance Indicators	Weightage %
	Process related (30 Marks)	60%
1.	Debugging ability	20%
2.	Correctness of Program codes	30%
3.	Quality of output achieved (LLO mapped)	10%
	Product related : 20 Marks	40%
4.	Completion and submission of practical in time	20%
5.	Answer to sample questions	20%
	Total (50 Marks)	100%

List of Student /Team Members

- (1) Farth Rande (2) Shivam Jannik
 (3) _____ (4) _____

Marks Obtained			Dated signature of Teacher
Process Related (30)	Product Related (20)	Total (50)	

Q2) Continue :

```

    int inversionCount = insertionSortAndCount(arr, n);
    printf("Sorted array: \n");
    printfArray(arr, n);
    printf("Number of inversions: %d \n", inversionCount);
    return 0;
}

```

PRQ: Q1) Program -

Steps to Modify Insertion Sort :

- i) Use Binary Search to find the correct position of the current element in the sorted portion of the array
- ii) Shift elements in the sorted portion to make space for the current element.
- iii) Insert the elements at its correct position.

Program :

```
#include <stdio.h>
```

// Function to perform Binary Search to find the correct position

```

int binarySearch(int arr[], int item, int low, int high)
{
    while (low <= high)

```

```

    {
        int mid = low + (high - low) / 2 ;

```

```
        if (item == arr[mid])
```

```
            return mid + 1;
```

```
        else if (item > arr[mid])
```

```
            low = mid + 1;
```

```
        else
```

```
            high = mid - 1;
```

```

    }
    return low;
}

```

```

}

```

// Function to perform Insertion Sort using Binary Search

void insertionSortBinarySearch(int arr[], int n)

{
 for (int i = 1; i < n; i++)

 {
 int key = arr[i];
 int j = i - 1;

 // Find the correct position using binary search

 int loc = binarySearch(arr, key, 0, j);

 // Shift elements to the right to create the correct position

 while (j >= loc) {
 arr[j + 1] = arr[j];
 j--;

 }
 arr[loc] = key;
}

}

// Function to perform Standard Insertion Sort

void insertionSort(int arr[], int n)

{
 for (int i = 1; i < n; i++)

 {
 int key = arr[i];
 int j = i - 1;

 // Move elements of arr[0...i-1], that are greater than key,

 // to one position ahead of their current position

 while (j >= 0 && arr[j] > key)

Q) Continue :

PR-10(2)

```
{
    arr[j+1] = arr[j];
    j = j - 1;
}
arr[j+1] = key;
}
```

// Function to print an array

```
void printArray (int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

// Function to compare performance

```
void comparePerformance ()
{
    int arr1[] = {12, 11, 13, 5, 6};
    int arr2[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr1) / sizeof(arr1[0]);
```

// Standard Insertion Sort

```
printf("Original array for standard insertion sort:\n");
printArray(arr1, n);
insertionSort(arr1, n);
printf("Sorted array using standard insertion sort:\n");
printArray(arr1, n);
```

// Modified Insertion Sort using Binary Search

```
printf("Original array for binary search insertion  
sort: \n");
```

```
PrintArray(arr2, n);
```

```
insertionSortBinarySearch(arr2, n);
```

```
printf("Sorted array using binary search insertion sort: \n");
```

```
PrintArray(arr2, n);
```

```
}
```

// Main Function

```
int main()
```

```
{
```

```
ComparePerformance();
```

```
return 0;
```

```
}
```

CAMPUSIFY

**✓ Practical No.12 : Write a 'C' Program to Implement
Singly Linked List with Operations : (i) Insert at
beginning, (ii) Search, (iii) Display**

(I) Practical Significance

A **Singly Linked List** is a linear data structure in which the elements are not stored in contiguous memory locations and each element is connected only to its next element using a pointer.

(II) Industry/ Employer Expected Outcome

Through this practical student should

1. Provide a reliable and efficient data structure that can be used as a building block for more complex algorithms and applications.
2. Perform basic operations such as insertion, deletion, and traversal without errors or unexpected behavior.
3. Minimize memory usage.
4. Optimize traversal and search operations

(III) Course Level Learning Outcomes

Implement basic operations on Linked List.

(IV) Laboratory Learning Outcome

Create Singly Linked List

(V) Relevant Affective domain related Outcome(s)

1. Follow safety practices.
2. Follow ethical practices.

(VI) Relevant Theoretical Background

Linked List can be defined as collection of objects called nodes that are randomly stored in the memory. A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.

The last node of the list contains pointer to the null. Uses of Linked List

The list is not required to be contiguously present in the memory. The node can reside anywhere in the memory and linked together to make a list. This achieves optimized utilization of space.

list size is limited to the memory size and doesn't need to be declared in advance.

Empty node cannot be present in the linked list.

We can store values of primitive types or objects in the singly linked list. Understanding Node Structure:

In a singly linked list, each node consists of two parts: data and a pointer to the next node. The data part stores the actual information, while the pointer (or reference) part stores the address of the next node in the sequence. This structure allows nodes to be dynamically linked together, forming a chain-like sequence.

Algorithm :**Insert at Beginning:**

- Create a new node
- Assign the new node's data with the input value
- Set the new node's next pointer to the Current head of the list
- Update the head to point to the new node

Search for a Value:

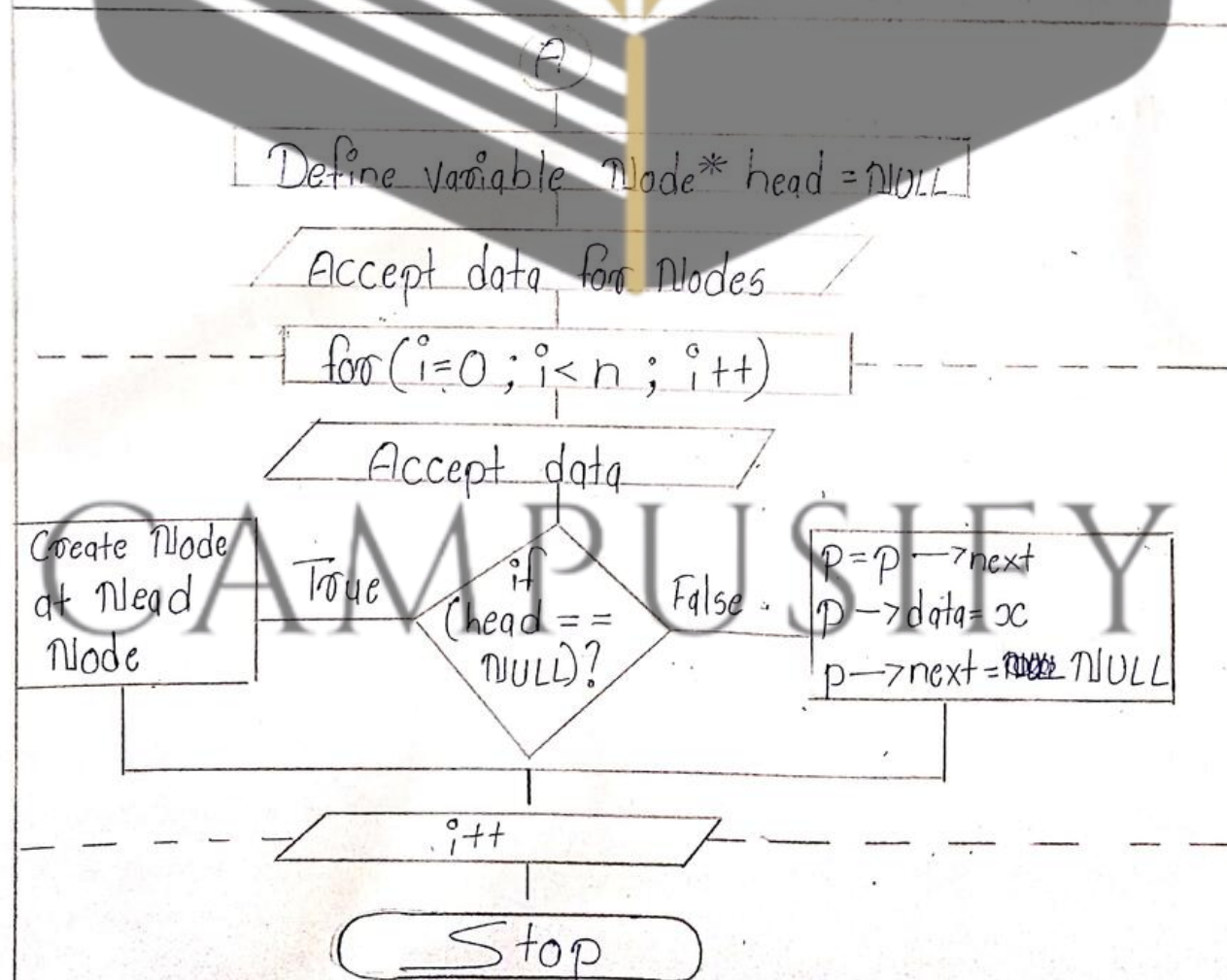
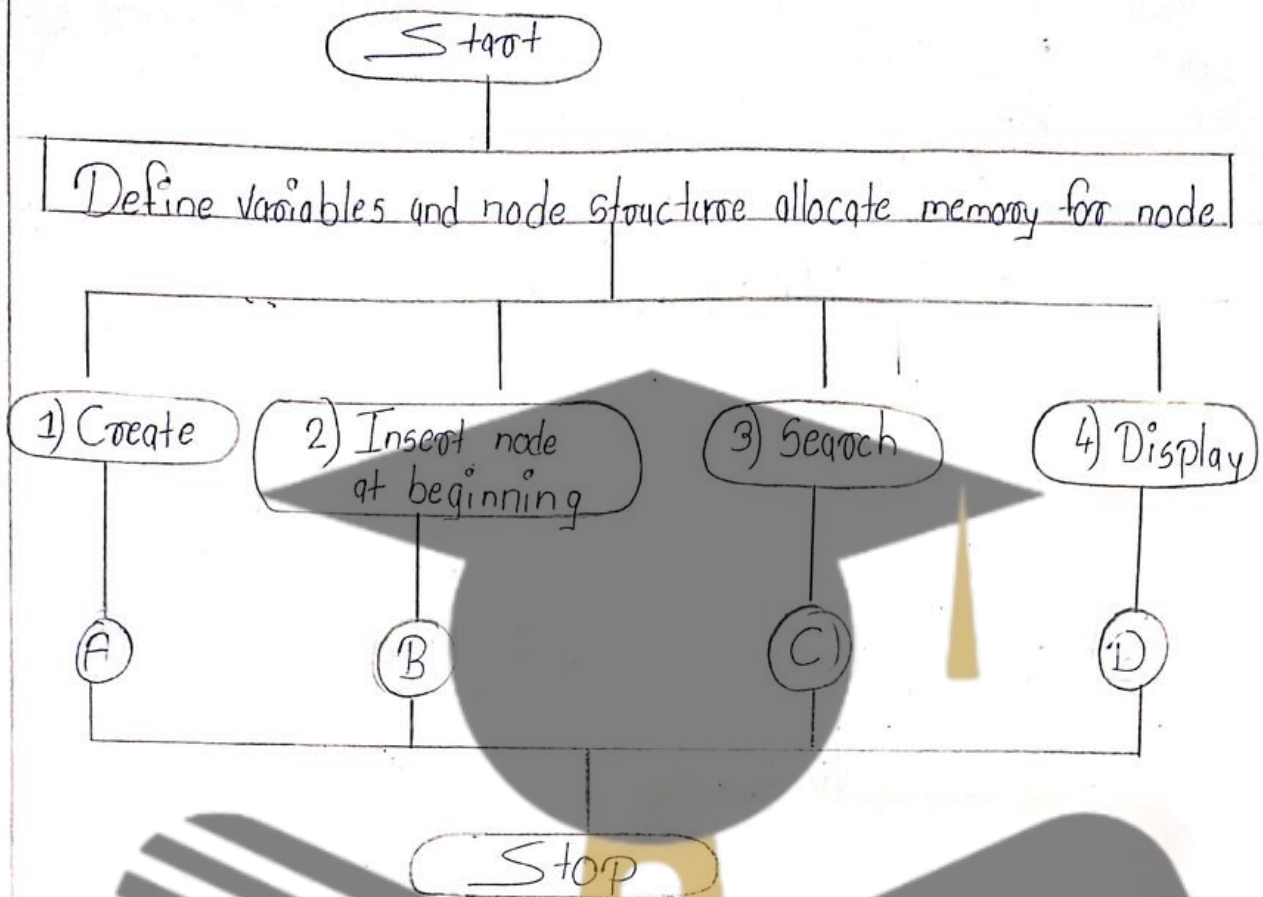
- Start from the head node
- Traverse the list while the current node is not NULL.
- If the current node's data matches the search key, return found.
- Move to the next node
- If the end of the list is reached without finding the key, return not found

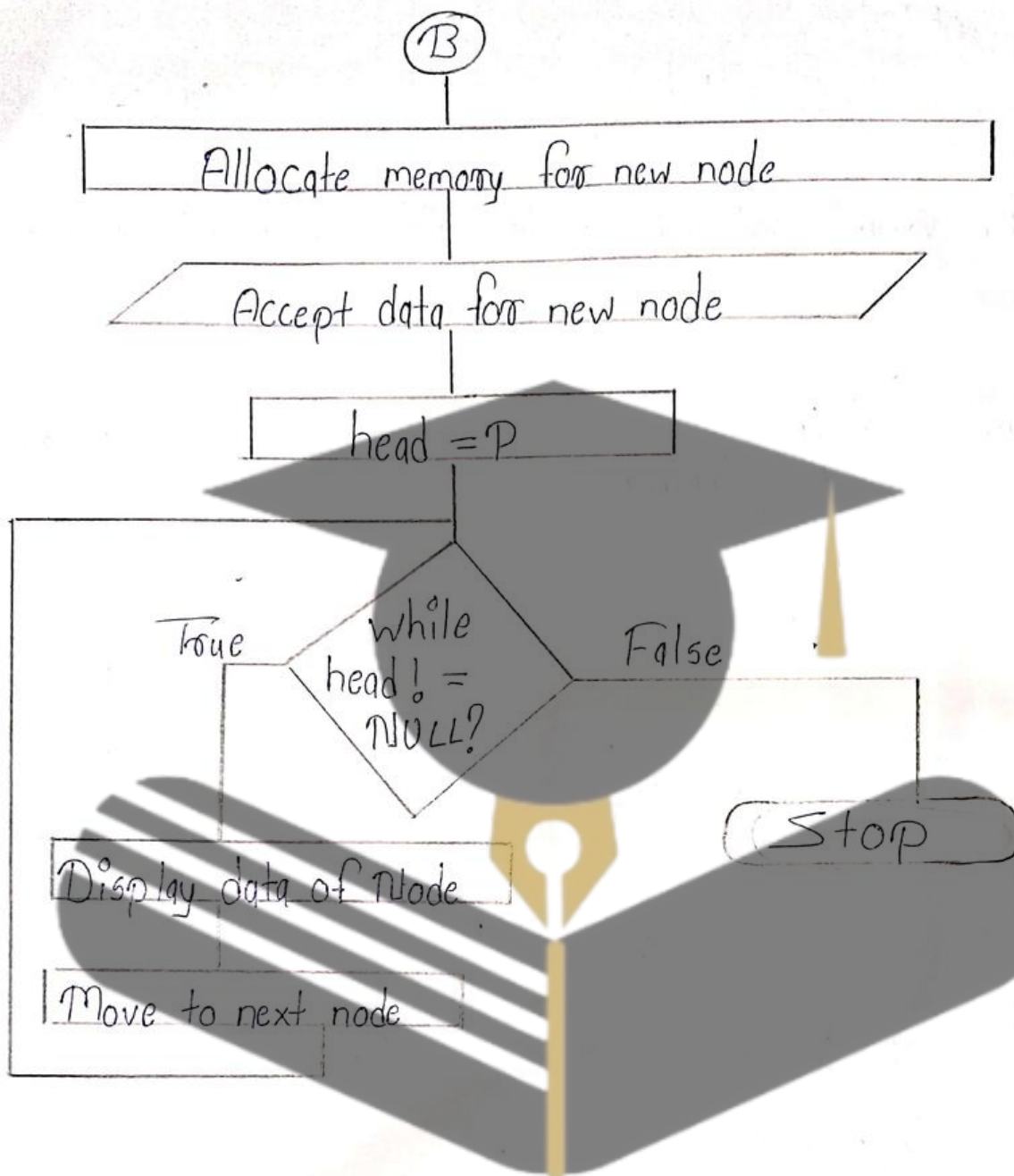
Display the List:

- Start from the head node.
- Traverse the list while the current node is not NULL
- Print the current node's data.
- Move to the next node
- End when the current node is NULL (indicating the end of the list)

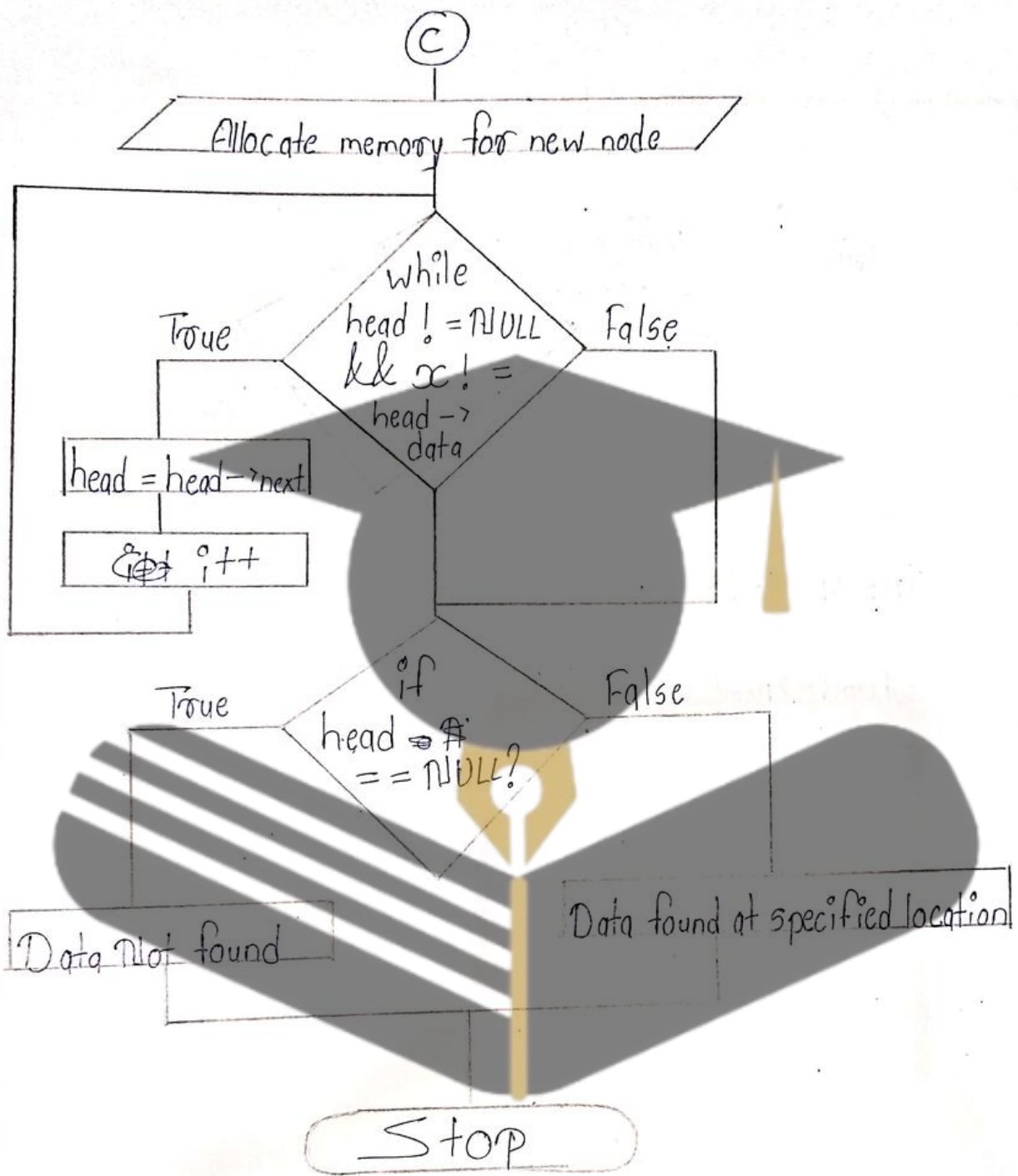
Flowchart :

PR-12

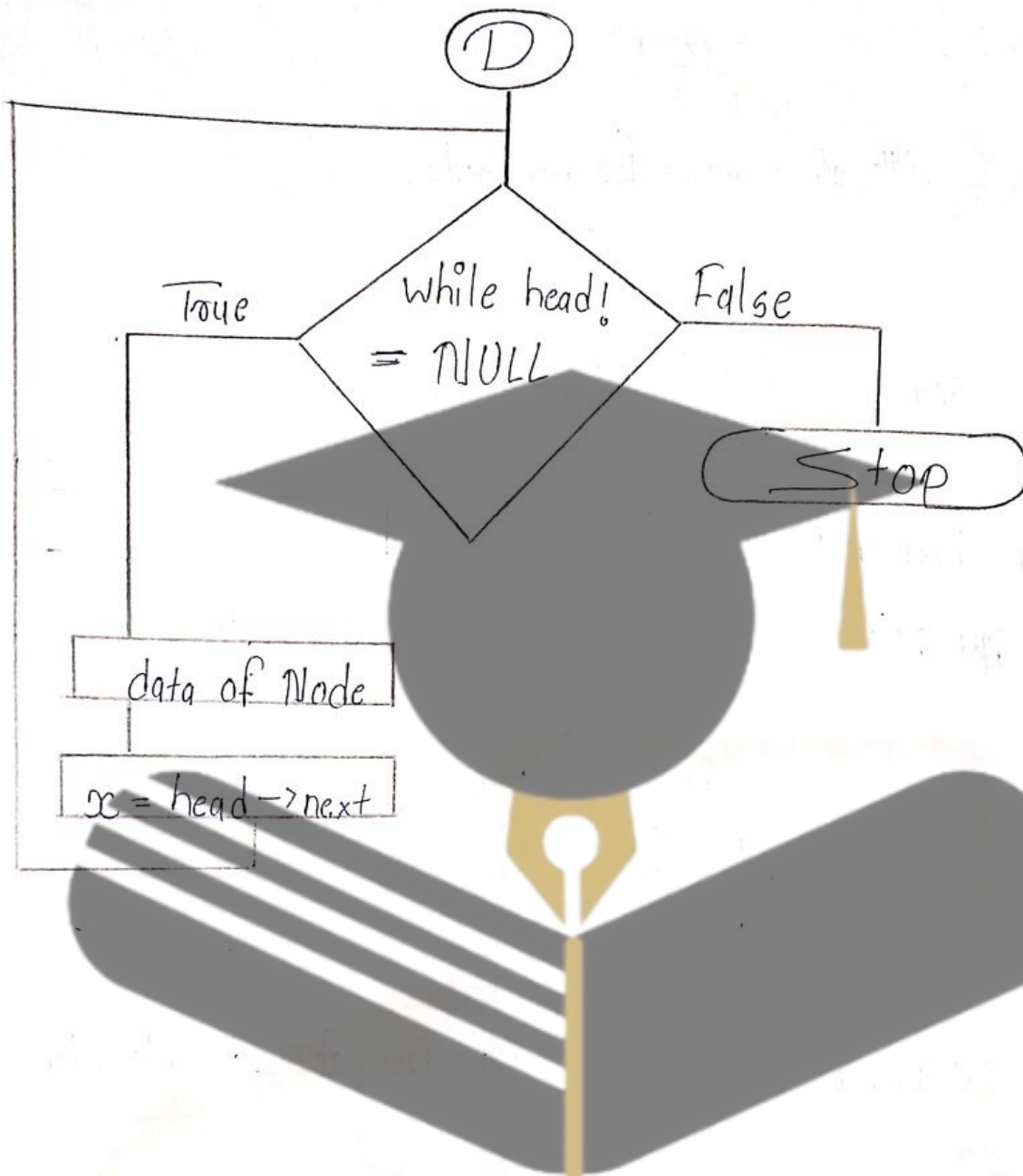




CAMPUSIFY



CAMPUSIFY



CAMPUSIFY

(IX) C Program Code

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

Void insertAtBeginning (struct Node** head-ref,
                        int new-data) {
    struct Node* new-node = (struct Node*) malloc
        (size of (struct Node));
    new-node->data = new-data;

```

(X) Resources required

Sr. No.	Equipment Name with broad specification	Relevant LLO Number
1	Computer System with all necessary Peripherals and Internet Connectivity 'C' Compiler / GCC Compiler/ Online 'C' Compiler All	ALL

(XI) Precautions to be Followed

1. Ensure that all C statements must end with a semicolon (;).
2. Use white spaces in c to describe blanks and tabs.
3. Ensure use of Dynamic memory allocation to store Stack data
4. Follow safety practices.

(XII) Result (Output of Program)

Linked List: 40 → 30 → 20 → 10 → NULL
 Element 20 found in the list.
 Final Linked List : 40 → 30 → 20 → 10 → NULL

C Program Code : Continue -

```
new_node -> next = (*head_ref);  
*head_ref = new_node;
```

```
}
```

```
int search(struct Node* head, int key)  
{
```

```
    struct Node* current = head;
```

```
    while (current != NULL) {
```

```
        if (current -> data == key) {
```

```
            return 1;
```

```
        }
```

```
        current = current -> next;
```

```
    }
```

```
    return 0;
```

```
}
```

```
void display(struct Node* node) {
```

```
    while (node != NULL) {
```

```
        printf("%d -> ", node -> data);
```

```
        node = node -> next;
```

```
    }
```

```
    printf("NULL\n");
```

```
}
```

```
int main()
```

```
{
```

```
    struct Node* head = NULL;
```

```
    insertAtBeginning(&head, 10);
```

```
    insertAtBeginning(&head, 20);
```

```
insertAtBeginning (& head, 30);  
insertAtBeginning (& head, 40);  
printf ("Linked List :");  
display (head);  
int key = 20;  
if (Search (head, key)) {  
printf ("Element %d found in the list.\n", key);  
} else {  
printf ("Element %d not found in the list.\n", key);  
}  
printf ("Final Linked List :");  
display (head);  
return 0;  
}
```

CAMPUSIFY

(XIII) Conclusion

We successfully completed Data structure Using C (313301) Practical No. 12 and Write a 'C' Program to Implement Singly Linked List with Operations : i) Insert at beginning ii) Search , iii) Display

(XIV) Practical Related Questions

1. Write a function to insert a node at the beginning of a Singly Linked List.
2. Write a function to detect if a singly linked list has a cycle. If a cycle is detected, return the starting node of the cycle.

(XV) Exercise

1. Explain the use of Singly Linked List
2. Write advantages and disadvantages of Singly Linked List
3. Calculate length of Singly Linked List

[Space to Write for Answers]

PRQ : Q1) Program -

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)
    malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

```

void insertAtBeginning (struct Node ** head_ref, int new_data)
{
    struct Node* new_node = createNode (new_data);
    new_node->next = *head_ref;
    *head_ref = new_node;
}

```

```

void printList (struct Node * head)
{

```

```

    struct Node * temp = head;
    while (temp != NULL)
    {

```

```

        printf ("%d -> ", temp->data);
        temp = temp->next;
    }
}

```

(XVI) **References / Suggestions for further Reading Software/Learning Websites**

1. Introduction to Singly Linked List - [GeeksforGeeks](#)
2. Linked List (Data Structures) - [javatpoint](#)

(XVII) **Assessment Scheme**

Performance Indicators		Weightage %
Process related (30 Marks)		60%
1.	Debugging ability	20%
2.	Correctness of Program codes	30%
3	Quality of output achieved (LLO mapped)	10%
Product related : 20 Marks		40%
4	Completion and submission of practical in time	20%
5	Answer to sample questions	20%
Total (50 Marks)		100%

List of Student /Team Members

(1) Parth Pande
 (3)

(2) Shivam Jamnik
 (4)

Marks Obtained			Dated signature of Teacher
Process Related (30)	Product Related (20)	Total (50)	

PRQ: Q1) Continue -

PR-12(1)

PRQ: Q1) Continue -

```
printf("NULL\n");
}
int main()
{
    struct Node* head = NULL;
    // Inserting nodes at the beginning
    insertAtBeginning(&head, 10);
    insertAtBeginning(&head, 20);
    insertAtBeginning(&head, 30);
    // Printing the list
    printList(head); // Output: 30 -> 20 -> 10 -> NULL
    return 0;
}
```

PRQ : Q2) Program :

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node
```

```
{
```

```
    int data
```

```
    struct Node* next;
```

```
};
```

```
struct Node* createNode(int data)
```

```
{
```

```
    struct Node* newNode = (struct Node*) malloc
    (sizeof(struct Node));
```

```
    newNode->data = data;
```

```
    newNode->next = NULL;
```

```
    return newNode;
```

```
}
```

```

struct Node* detectCycle(struct Node* head)
{
    struct Node* slow = head, *fast = head;
    while (fast != NULL && fast->next != NULL)
    {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            break;
        }
    }
    if (fast == NULL || fast->next == NULL)
    {
        return NULL;
    }
    slow = head;
    while (slow != fast)
    {
        slow = slow->next;
        fast = fast->next;
    }
    return slow;
}

void printList(struct Node* head)
{
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

PRQ: Q2) Continue -

```
int main()
{
    struct Node * head = createNode(1);
    head -> next = createNode(2);
    head -> next -> next = createNode(3);
    head -> next -> next -> next = createNode(4);
    head -> next -> next -> next -> next = createNode(5);

    // Creating a Cycle: pointing the next of the last node to the
    // second node
    head -> next -> next -> next -> next -> next =
        head -> next;

    // Detecting Cycle
    struct Node * cycle_start = detectCycle(head);
    if (cycle_start != NULL) {
        printf("Cycle detected at node with data: %d \n",
            cycle_start -> data);
    } else {
        printf("No cycle detected. \n");
    }
    return 0;
}
```

Practical No. 13 : Write a 'C' Program to Implement Singly Linked List with Operations : (i) Insert at end (ii) Insert after (iii) Delete (iv) Display

(I) Practical Significance

A **Singly Linked List** is a linear data structure in which the elements are not stored in contiguous memory locations and each element is connected only to its next element using a pointer.

(II) Industry/ Employer Expected Outcome

Through this practical student should

1. Provide a reliable and efficient data structure that can be used as a building block for more complex algorithms and applications.
2. Perform basic operations such as insertion, deletion, and traversal without errors or unexpected behavior.
3. Minimize memory usage
4. Optimize traversal and search operations

(III) Course Level Learning Outcomes

Implement basic operations on Linked List.

(IV) Laboratory Learning Outcome

Create Singly Linked List

(V) Relevant Affective domain related Outcome(s)

1. Follow safety practices.
2. Follow ethical practices.

(VI) Relevant Theoretical Background

Linked List can be defined as collection of objects called **nodes** that are randomly stored in the memory.

A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.

The last node of the list contains pointer to the null.

Uses of Linked List

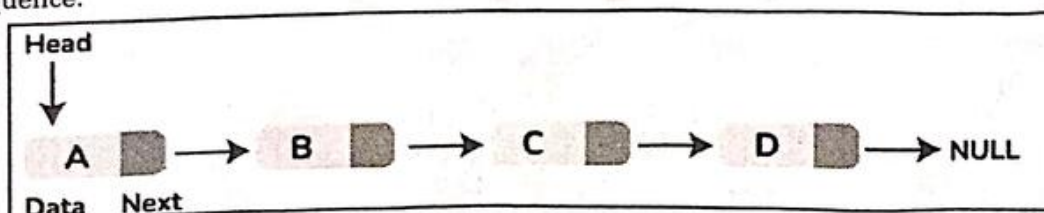
The list is not required to be contiguously present in the memory. The node can reside any where in the memory and linked together to make a list. This achieves optimized utilization of space.

list size is limited to the memory size and doesn't need to be declared in advance. Empty node cannot be present in the linked list.

We can store values of primitive types or objects in the singly linked list.

Understanding Node Structure:

In a singly linked list, each node consists of two parts: data and a pointer to the next node. The data part stores the actual information, while the pointer (or reference) part stores the address of the next node in the sequence. This structure allows nodes to be dynamically linked together, forming a chain-like sequence.



Algorithm :

1) Insert at End

- Input: head, data

- Steps :

- i) Create a new node with data

- ii) If head is NULL, set head to new node

- iii) Else, traverse to the last node and set its next to the new node

2) Insert After a Specific Node

- Input: prevNode, data

- Steps :

- i) If prevNode is NULL, return

- ii) Create a new node with data

- iii) Set new node's ~~next~~ next to prevNode $\rightarrow$ next, then set prevNode $\rightarrow$ next to new node.

~~(or) Else return~~

3) Delete a Node

- Input: head, Key

- Steps :

- i) If head $\rightarrow$ data equals key, set head to head $\rightarrow$ next and free the old head.

- ii) Else, traverse the list to find the node with key, keeping track of the previous node

- iii) If found, unlink and free the node

Algorithm: Continue -

4) Display List

- Input: head

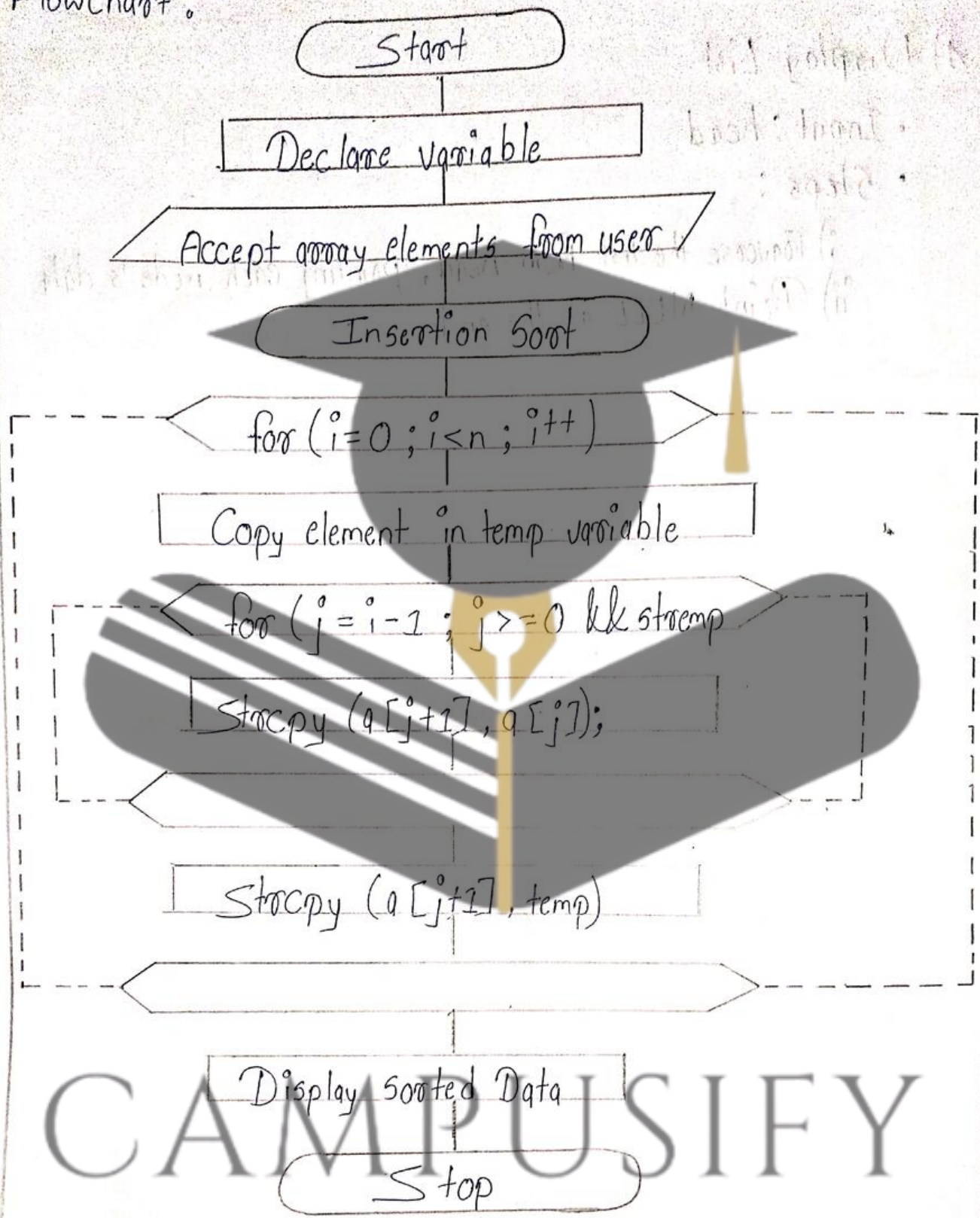
- Steps:

- i) Traverse the list from head, printing each node's data
- ii) Print NULL at the end



CAMPUSIFY

Flowchart:



(VIII) C Program Code

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data)
{
    struct Node* newNode = (struct Node*) malloc (sizeof
    (struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

void insertAtEnd(struct Node** head, int data)
{
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
}
```

C Program Code : Continue :

```

    temp -> next = newNode ;
}
void insertAfter(struct Node* prevNode, int data)
{
    if (prevNode == NULL) {
        printf("Previous node cannot be NULL.\n");
        return ;
    }
    struct Node* newNode = createNode(data);
    newNode -> next = prevNode -> next;
    prevNode -> next = newNode ;
}
void deleteNode(struct Node** head, int key)
{
    struct Node* temp = *head;
    struct Node* prev = NULL;
    if (temp != NULL && temp -> data == key) {
        *head = temp -> next;
        free(temp);
        return ;
    }
    while (temp != NULL && temp -> data != key) {
        prev = temp;
        temp = temp -> next;
    }
    if (temp == NULL) {
        printf("Key %d not found in the list.\n", key);
        return ;
    }
    prev -> next = temp -> next;
    free(temp);
}

```

```

void displayList(struct Node* node) {
    while (node != NULL) {
        printf("%d -> ", node->data);
        node = node->next;
    }
    printf("NULL\n");
}

int main()
{
    struct Node* head = NULL;
    insertAtEnd(&head, 10);
    insertAtEnd(&head, 20);
    insertAtEnd(&head, 30);
    insertAtEnd(&head, 40);

    printf("Linked List after inserting at the end:\n");
    displayList(head);

    struct Node* temp = head->next;
    insertAfter(temp, 25);

    printf("\n Linked List after inserting 25 after 20:\n");
    displayList(head);

    deleteNode(&head, 30);

    printf("\n Linked list after deleting node with data 30:\n");
    displayList(head);

    return 0;
}

```

(IX) Resources required

Sr. No.	Equipment Name with broad specification	Relevant LLO Number
1	Computer System with all necessary Peripherals and Internet Connectivity 'C' Compiler / GCC Compiler/ Online 'C' Compiler All	ALL

(X) Precautions to be Followed

1. Ensure that all C statements must end with a semicolon (;).
2. Use white spaces in c to describe blanks and tabs.
3. Ensure use of Dynamic memory allocation to store Stack data
4. Follow safety practices.

(XI) Result (Output of Program)

Linked List after insertion at end :

10 → 20 → 30 → 40 → NULL

Linked List after inserting 25 after 20;

10 → 20 → 25 → 30 → 40 → NULL

Linked List after deleting 30;

10 → 20 → 25 → 40 → NULL

(XII) Conclusion

We completed 313301 DSU Practical No. 13 in which we write a 'C' program to Implement Singly Linked with Operations : (i) Insert at end ii) Insert after
iii) Delete iv) Display.

(XIII) Practical Related Questions

1. Write a function to insert a node at the end in a Singly Linked List.
2. Write a function to check whether a singly linked list is a palindrome or not.

(XIV) Exercise

1. Write applications of Singly Linked List.
2. Explain the procedure to find middle of singly linked list.

[Space to Write for Answers]

PRQ: Q1) Program -

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*) malloc
        (sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
```

```

void printList(struct Node *head) {
    struct Node *temp = head;
    while (temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

(XV) References / Suggestions for further Reading Software/Learning Websites

1. Introduction to Singly Linked List - GeeksforGeeks
2. Linked List (Data Structures) - javatpoint

(XVI) Assessment Scheme

	Performance Indicators	Weightage %
	Process related (30 Marks)	60%
1.	Debugging ability	20%
2.	Correctness of Program codes	30%
3.	Quality of output achieved (LLO mapped)	10%
	Product related : 20 Marks	40%
4.	Completion and submission of practical in time	20%
5.	Answer to sample questions	20%
	Total (50 Marks)	100%

List of Student /Team Members

- (1) Prath Pandey (2) Shivam Jamnik
 (3) _____ (4) _____

Marks Obtained			Dated signature of Teacher
Process Related (30)	Product Related (20)	Total (50)	

PRQ: 01) Continue -

01)

```
int main()
{
    struct Node* head = NULL;
    insertAtEnd(&head, 10);
    insertAtEnd(&head, 20);
    insertAtEnd(&head, 30);
    insertAtEnd(&head, 40);
    printList(head);
    return 0;
}
```

PRQ: 02) Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
```

```
struct Node {
    int data;
    struct Node* next;
};
```

```
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*) malloc
(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

```
void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
```

```

if (*head == NULL) {
    *head = newNode;
    return;
}

```

```

struct Node * temp = *head;
while(temp -> next != NULL) {
    temp = temp -> next;
}
temp -> next = newNode;
}

```

```

struct Node * reverseList(struct Node* head) {
    struct Node * prev = NULL;
    struct Node * current = head;
    struct Node * next = NULL;
    while (current != NULL) {
        next = current -> next;
        current -> next = prev;
        prev = current;
        current = next;
    }
    return prev;
}

```

```

bool isPalindrome(struct Node* head) {
    if (head == NULL || head -> next == NULL) {
        return true;
    }
}

```

```

struct Node* slow = head;
struct Node* fast = head;
while (fast != NULL && fast -> next != NULL) {
    slow = slow -> next;
    fast = fast -> next -> next;
}

```

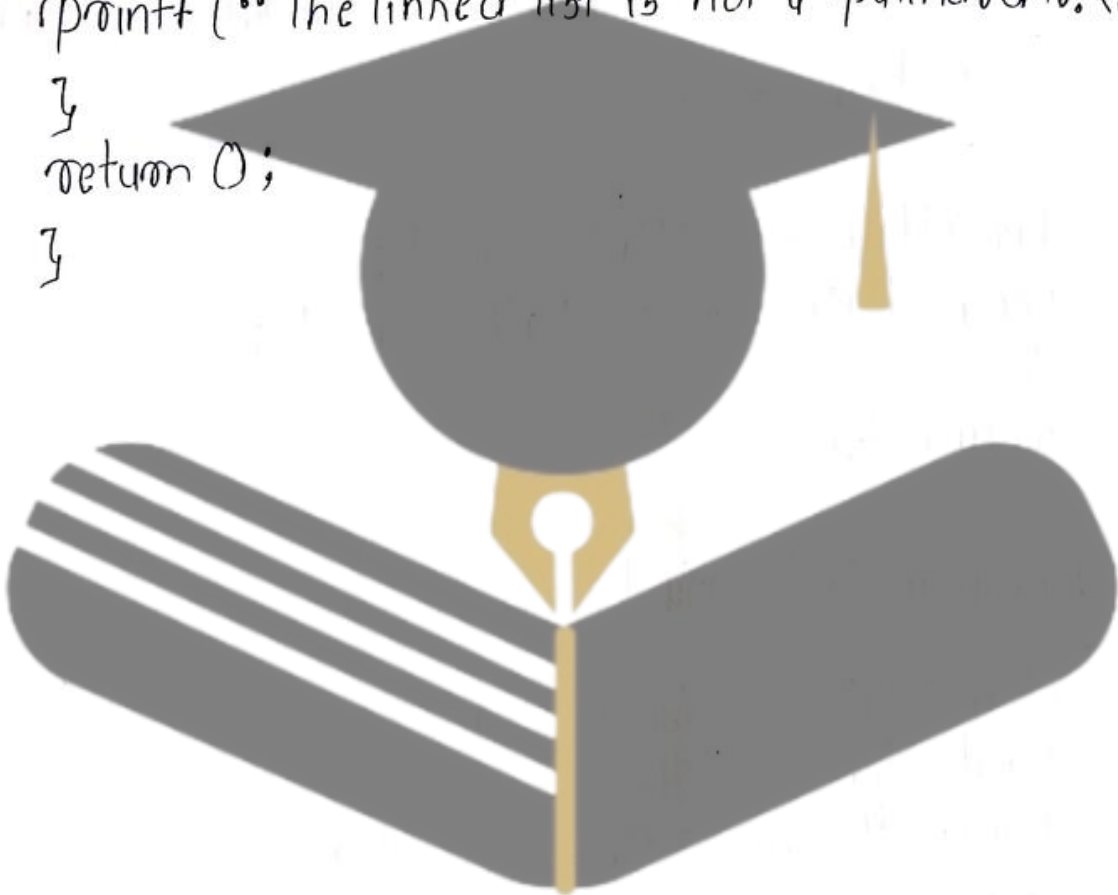
PRQ : Q2) Continue :

```
struct Node * secondHalf = reverseList (slow);
struct Node * firstHalf = head;
while (secondHalf != NULL) {
    if (firstHalf->data != secondHalf->data)
    {
        return false;
    }
    firstHalf = firstHalf->next;
    secondHalf = secondHalf->next;
}
return true;
}

void printList (struct Node * head)
{
    struct Node * temp = head;
    while (temp != NULL) {
        printf ("%d -> ", temp->data);
        temp = temp->next;
    }
    printf ("NULL \n");
}

int main ()
{
    struct Node * head = NULL;
    insertAtEnd (&head, 1);
    insertAtEnd (&head, 2);
    insertAtEnd (&head, 3);
    insertAtEnd (&head, 2);
    insertAtEnd (&head, 1);
}
```

```
if (isPalindrome(head))  
{  
    printf("The linked list is a palindrome.\n");  
}  
else else {  
    printf("The linked list is not a palindrome.\n");  
}  
return 0;  
}
```



CAMPUSIFY

✓ Practical No. 16 : Write a 'C' Program to Perform PUSH and POP Operations on Stack using an Array

(I) Practical Significance

Stacks are a fundamental data structure in computer science and have numerous practical applications based on their LIFO (Last In, First Out) property. Hence Stack is Linear Data Structure where insertion and deletion take place only from one end called as Stack Top.

(II) Industry/ Employer Expected Outcome

Through this practical student should

1. Understand Stack as ADT
2. Represent Stack in memory using Array.
3. Write an algorithm to perform PUSH and POP operations on Stack.
4. Develop simple C program to implement stack operation using Array.
5. Save/Compile/ Debug/ the C program.
6. Check for the desired output.

III) Course Level Learning Outcomes

Demonstrate Stack operation using Array.

IV) Laboratory Learning Outcome

Perform Operations on the Stack using the Array.

V) Relevant Affective domain related Outcome(s)

1. Follow safety practices.
2. Follow ethical practices.

VI) Relevant Theoretical Background

A stack is an ordered list in which insertion and deletion are done at one end, called top. The last element inserted is the first one to be deleted. Hence, it is called the Last in First out (LIFO) or First in Last out (FILO) list. Two major operations through which changes can be made are PUSH and POP. When an element is inserted in a stack, the concept is called push, and when an element is removed from the stack, the concept is called pop. Trying to pop out an empty stack is called underflow and trying to push an element in a full stack is called overflow.

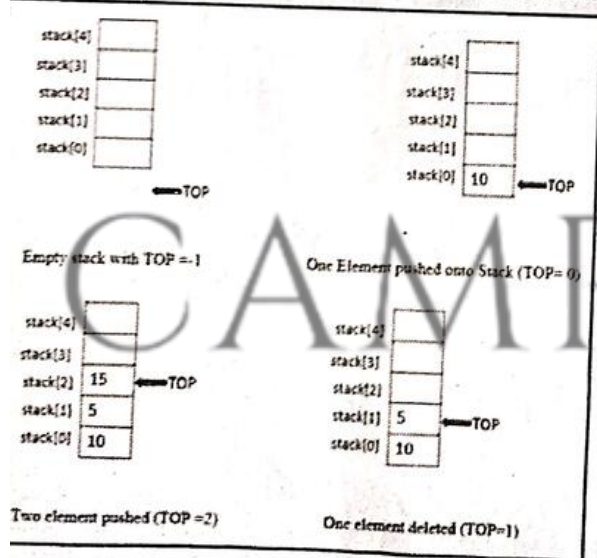


Figure 16.1 : Stack PUSH and POP operations

ashira State Board of Technical Education ('K' scheme)

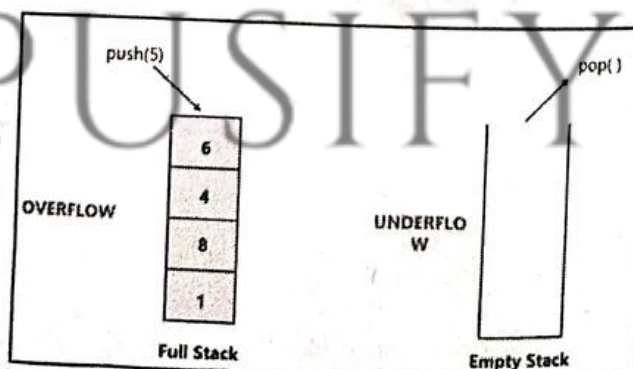


Figure 16.2 : Stack Overflow and Underflow

(VII) Algorithm :**Initialize :**

- Define a constant MAX to specify the maximum size of the stack.
- Initialize an array stack [MAX] to hold stack elements
- Set a variable top to -1 to keep track of the top of the stack

Push Operation :

- Input : Element value to push onto the stack
- Check : If top is equal to MAX - 1, output "Stack Overflow".
- Else
 - Increment top by 1.
 - Assign value to stack [top]
 - Output that value has been pushed onto the stack

(VIII) Flow Chart**POP Operation :**

- Check : If top is equal to -1, output "Stack Underflow".
- Else :
 - Retrieve stack [top].
 - Decrement top by 1
 - Output the retrieved value as the popped element.

Algorithm : Continue -

Display Operation:

- Check : If top is equal to -1, output "Stack is empty".
- Else :
 - Loop from top to 0
 - Print each element from stack[top] to stack[0].

Main Function:

- Repeat :
 - Display a menu with options (Push, Pop, Display, ~~Exit~~)
 - Read user choice
 - Based on the choice, call the respective operation function
- ~~Until~~ Until the user chooses the exit.

CAMPUSIFY

(IX) C Program Code

```
#include <stdio.h>
#define MAX 5

int stack[MAX];
int top = -1;

void push(int value) {
    if (top == MAX - 1) {
        printf("Stack Overflow! Cannot push %d\n", value);
    } else {
        top++;
        stack[top] = value;
        printf("%d pushed onto the stack.\n", value);
    }
}

int pop() {
    if (top == -1) {
        printf("Stack Underflow! No element to pop.\n");
        return -1;
    } else {
        int value = stack[top];
        top--;
        printf("%d popped from the stack.\n", value);
    }
}
```

C Program Code :: Continue

```
        return value ;
    }
}

void display ( ) {
    if (top == -1) {
        printf ("Stack is empty.\n");
    } else {
        printf ("Stack elements are : \n");
        for (int i = top ; i >= 0 ; i--) {
            printf ("%d \n", stack[i]);
        }
    }
}

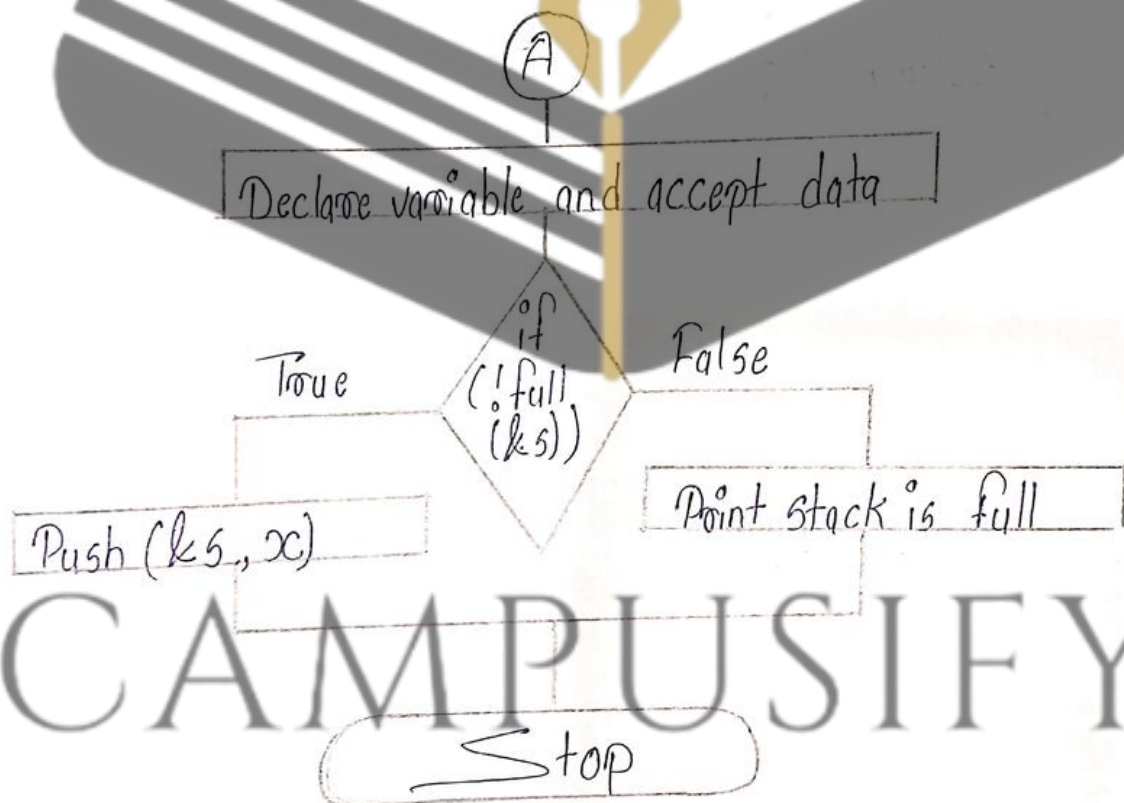
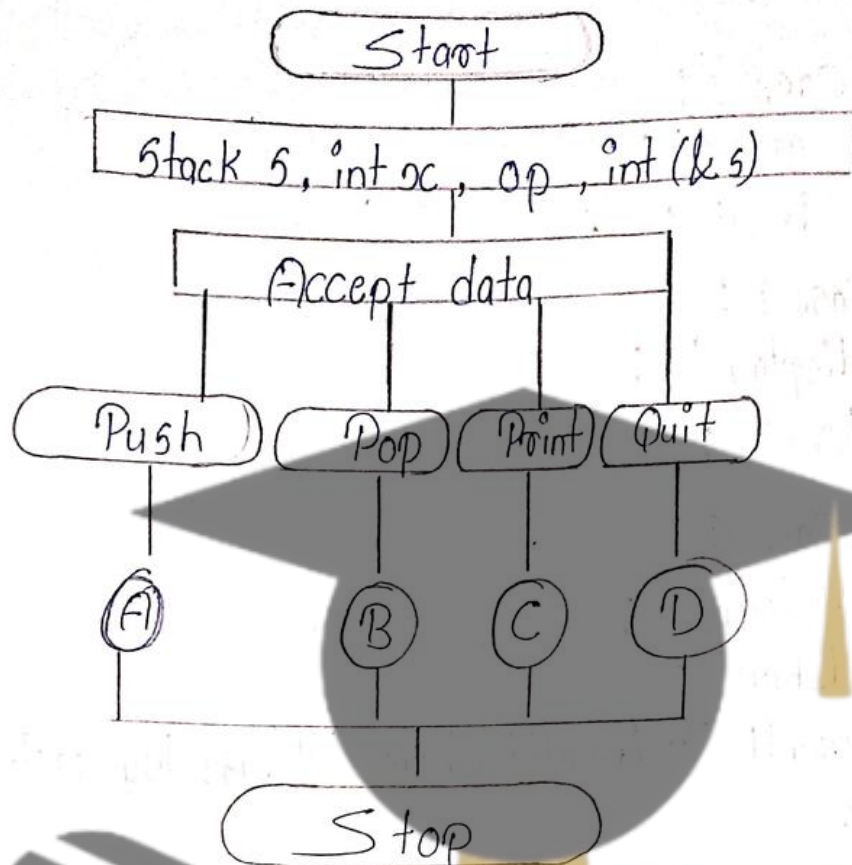
int main()
{
    int choice, value ;
    while (1) {
        printf ("\n -- Stack Operations -- \n");
        printf ("1. Push \n");
        printf ("2. Pop \n");
        printf ("3. Display \n");
        printf ("4. Exit \n");
        printf ("Enter your choice : ");
        scanf ("%d", &choice);
        switch (choice) {
            Case 1 :
                printf ("Enter the value to push : ");
                scanf ("%d", &value);
                push (value) ;
                break ;
        }
    }
}
```

C Program Code : Continue

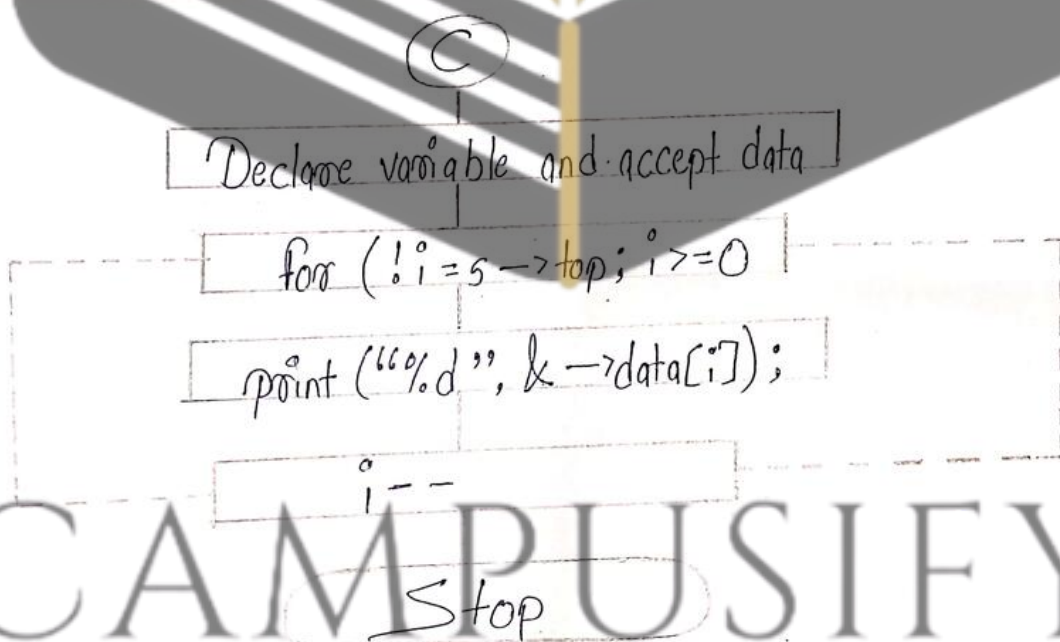
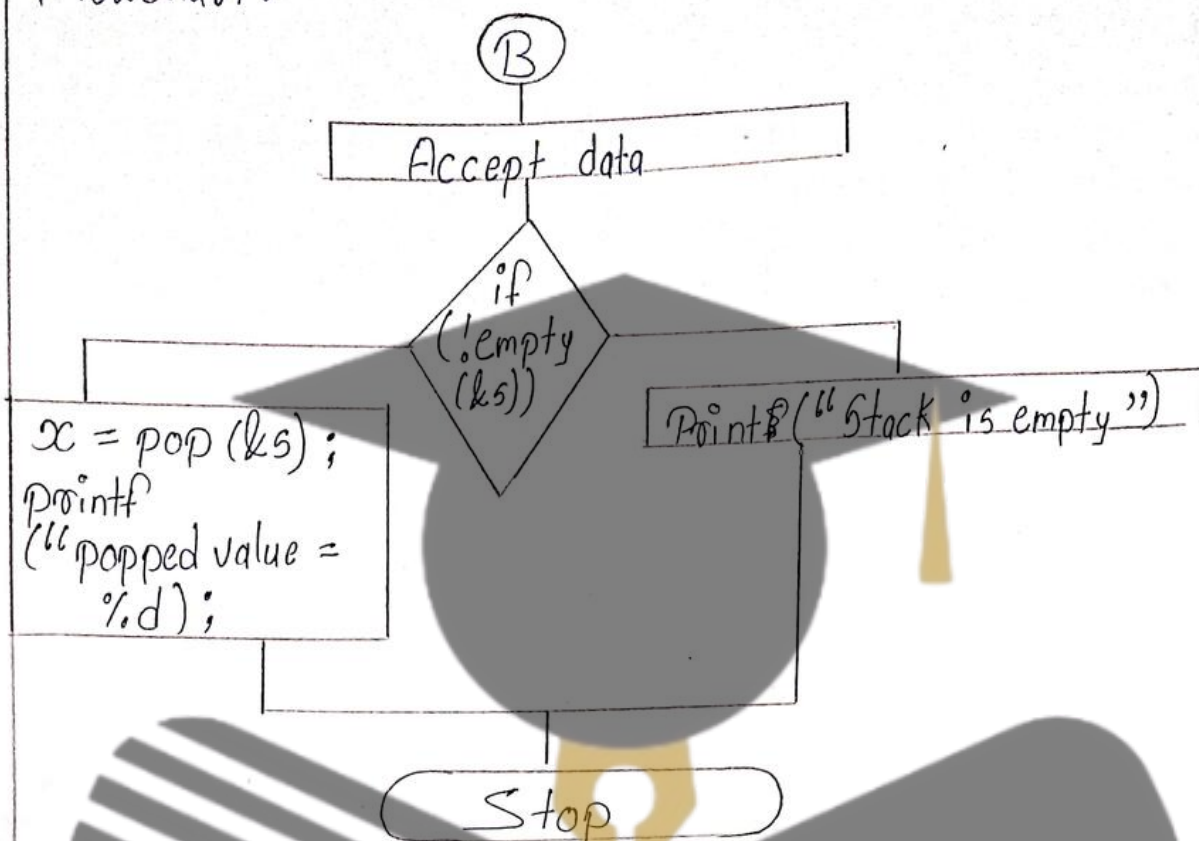
```
Case 2 :  
    pop();  
    break ;  
Case 3 :  
    display();  
    break ;  
Case 4 :  
    return 0 ;  
default :  
    printf ("Invalid Choice ! Please try again.\n");  
}  
}  
return 0;  
}
```

CAMPUSIFY

Flowchart



Flowchart:



(X) Resources required

Sr. No.	Equipment Name with broad specification	Relevant LLO Number
1	Computer System with all necessary Peripherals and Internet Connectivity 'C' Compiler / GCC Compiler/ Online 'C' Compiler All	ALL

(XI) Precautions to be Followed

1. Ensure that all C statements must end with a semicolon (;).
2. Use white spaces in c to describe blanks and tabs.
3. Ensure use of Static memory allocation using fixed memory size
4. Follow safety practices.

(XII) Result (Output of Program)

-- Stack Operations --

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1

Enter the value to push: 23

23 - Pushed onto the stack

-- Stack Operation

1. Push
2. Pop
3. Display
4. Exit

Enter your choice.

(XIII) Conclusion

We successfully completed Data Structure Using C (313301) Practical No.16 and write a 'C' program to perform PUSH and POP operations on stack using an Array.

(XIV) Practical Related Questions

- Write a C program to perform following operations on Empty stack:
(a) PUSH (10), PUSH (20), POP, PUSH (10), PUSH (20), POP, POP, POP, PUSH (20), POP.
- Write a C program to reverse a String using Stack.
- Write a C program to check whether the given string is palindrome or not using stack.

(XV) Exercise

- Enlist stack applications.
- Consider the following stack and observe how many operations are required to delete smallest element from given stack.

2
4
1
7
5
9

[Space to Write for Answers]

PRQ: Q1) Program:

#include <stdio.h>

#include <stdlib.h>

#define MAX 100

struct Stack {

int data[MAX];

int top;

};

void initStack(struct Stack* stack)

{

stack->top = -1;

}

int isEmpty(struct Stack* stack)

{

return stack->top == -1;

}

```

int isFull(struct Stack* stack)
{
    return stack->top == MAX - 1;
}

void push(struct Stack* stack, int value)
{
    if (isFull(stack))
    {
        printf("Stack overflow, cannot push %d\n",
            value);
        return;
    }
    stack->data[++(stack->top)] = value;
}

```

(XVI) References / Suggestions for further Reading Software/Learning Websites

<https://www.scholarhat.com/tutorial/datastructures/binary-search-tree-in-data-structures>

(XVII) Assessment Scheme

	Performance Indicators	Weightage %
	Process related (30 Marks)	60%
1.	Debugging ability	20%
2.	Correctness of Program codes	30%
3.	Quality of output achieved (LLO mapped)	10%
	Product related : 20 Marks	40%
4.	Completion and submission of practical in time	20%
5.	Answer to sample questions	20%
	Total (50 Marks)	100%

List of Student /Team Members

(1) (2)
 (3) (4)

Marks Obtained			Dated signature of Teacher
Process Related (30)	Product Related (20)	Total (50)	

PRQ: Q1) Continue -

```
printf("Pushed %d onto the stack.\n", value);
```

```
}
int pop(struct stack * stack)
```

```
{
    if (isEmpty(stack))
```

```
{
    printf("Stack underflow, cannot pop.\n");
    return -1;
}
```

```
int value = stack->data[(stack->top) - -];
printf("Popped %d from the stack.\n", value);
return value;
}
```

```
int main()
```

```
{
    struct stack stack;
    initStack(&stack);
```

```
    push(&stack, 10);
```

```
    push(&stack, 20);
```

```
    pop(&stack);
```

```
    push(&stack, 10);
```

```
    push(&stack, 20);
```

```
    pop(&stack);
```

```
    pop(&stack);
```

```
    pop(&stack);
```

```
    push(&stack, 20);
```

```
    pop(&stack);
```

```
    return 0;
```

```
}
```

PRQ: Q2) Program :

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define MAX 100
```

```
struct stack {
    int top;
    char data[MAX];
};
```

```
void initStack (struct Stack * stack)
```

```
{
    stack->top = -1;
}
```

```
int isFull (struct Stack * stack)
```

```
{
    return stack->top == MAX - 1;
}
```

```
int isEmpty (struct Stack * stack)
```

```
{
    return stack->top == -1;
}
```

```
void push (struct Stack * stack, char ch)
```

```
{
    if (isFull (stack))
```

```
{
    printf ("Stack overflow, cannot push %C \n", ch);
    return;
}
```

```
stack->data[++(stack->top)] = ch;
```

```
}
```

PRQ: Q2) Continue -

```
char pop(struct Stack * stack)
{
    if (isEmpty(stack))
    {
        printf("Stack underflow, cannot pop.\n");
        return '\0';
    }
    return stack->data[(stack->top)--];
}
```

```
void reverseString(char * str)
{
    int n = strlen(str);
    struct Stack stack;
    initStack(&stack);
    for (int i = 0; i < n; i++)
    {
        push(&stack, str[i]);
    }
    for (int i = 0; i < n; i++)
    {
        str[i] = pop(&stack);
    }
}
```

```
int main()
{
    char str[MAX];
    printf("Enter a string:");
    fgets(str, MAX, stdin);
    str[strlen(str)] = '\0';
    printf("Reversed string: %s\n", str);
    return 0;
}
```

PRQ: Q3) Program -

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define MAX 100
```

```
struct Stack {
    int top;
    char data [MAX];
};
```

```
void initstack(struct Stack* stack)
```

```
{
    stack->top = -1;
}
```

```
int isFull(struct Stack* stack)
```

```
{
    return stack->top == MAX - 1;
}
```

```
int isEmpty(struct Stack* stack)
```

```
{
    return stack->top == -1;
}
```

```
void push(struct Stack* stack, char ch)
```

```
{
    if (isFull(stack))
```

```
{
        printf("Stack overflow, cannot push %c\n", ch);
        return;
    }
```

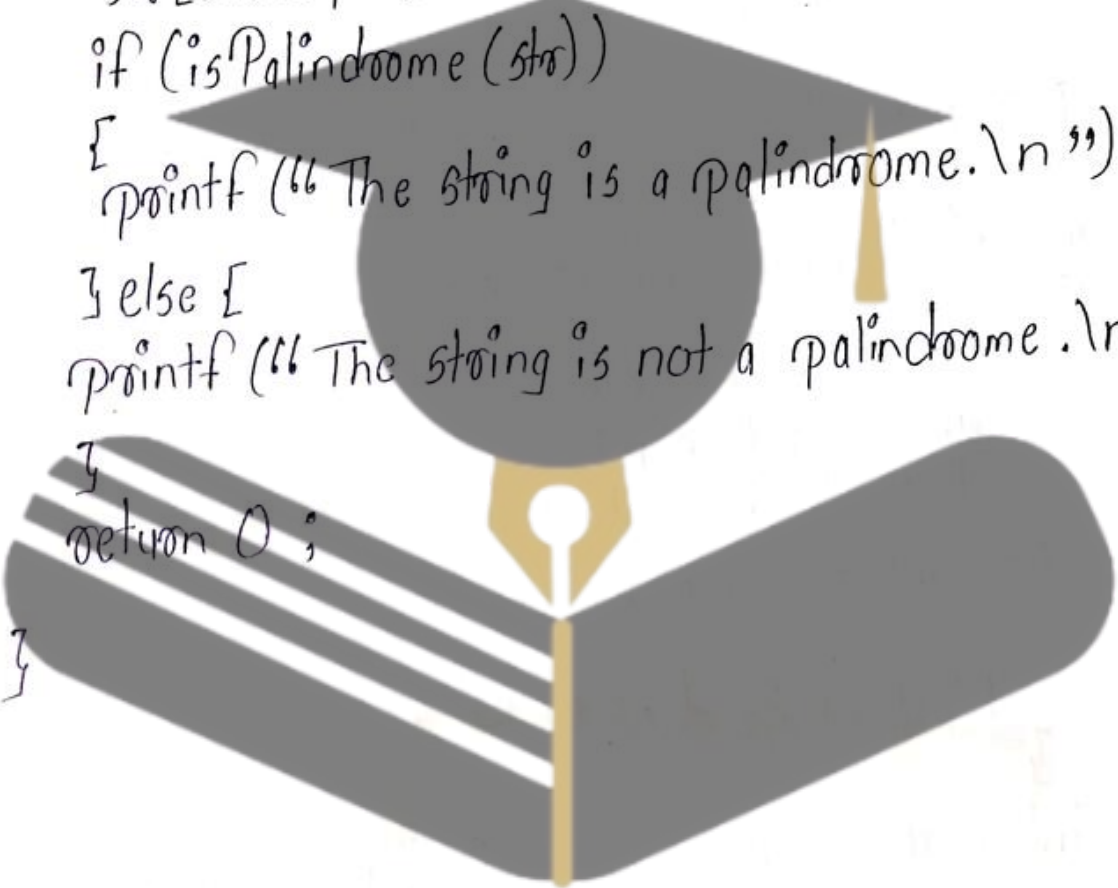
```
    stack->data[++(stack->top)] = ch;
}
```

```
}
```

```
char pop(struct Stack* stack)
{
    if (isEmpty(stack))
    {
        printf("Stack underflow, cannot pop.\n");
        return '\0';
    }
    return stack->data[(stack->top) - -];
}

int isPalindrome(char* str)
{
    int n = strlen(str);
    struct Stack stack;
    initStack(&stack);
    for (int i = 0; i < n / 2; i++)
    {
        push(&stack, str[i]);
    }
    int start = (n % 2 == 0) ? n / 2 : n / 2 + 1;
    for (int i = start; i < n; i++)
    {
        char topChar = pop(&stack);
        if (topChar != str[i])
        {
            return 0;
        }
    }
    return 1;
}
```

```
int main()
{
    char str[MAX];
    printf("Enter a string : ");
    fgets(str, MAX, stdin);
    str[strlen(str, "\n")] = 0;
    if (isPalindrome(str))
    {
        printf("The string is a palindrome.\n");
    } else {
        printf("The string is not a palindrome.\n");
    }
    return 0;
}
```



CAMPUSIFY

✓ Practical No. 17 : Write a 'C' Program to Perform PUSH and POP Operations on Stack using Linked List

(I) Practical Significance

Stacks are a fundamental data structure in computer science and have numerous practical applications based on their LIFO (Last In, First Out) property. Hence Stack is Linear Data Structure where insertion and deletion take place only from one end called as Stack Top.

(II) Industry/ Employer Expected Outcome

Through this practical student should,

1. Understand linked list structure
2. Represent stack in memory using Linked List.
3. Write an algorithm to perform PUSH and POP operations on Stack using Linked List.
4. Write simple C program to implement stack operation using Linked List.
5. Save/Compile/ Debug/ the C program.
6. Check for the desired output.

(III) Course Level Learning Outcomes

Demonstrate Stack operations using Linked List.

(IV) Laboratory Learning Outcome

Perform Operations on the Stack using a Linked List.

(V) Relevant Affective domain related Outcome(s)

1. Follow safety practices.
2. Follow ethical practices.

(VI) Relevant Theoretical Background

A stack is an ordered list in which insertion and deletion are done at one end, called top. The last element inserted is the first one to be deleted. Hence, it is called the Last in First out (LIFO) or First in Last out (FILO) list.

Stack using Linked List

Push operation: The first thing we need before pushing an element is to create a new node. Check if the stack is not already full. Now, we follow the same concept we learnt while inserting an element at the head or at the index 0 in a linked list. Just set the address of the current top in the next member of the new node, and update the top element with this new node.

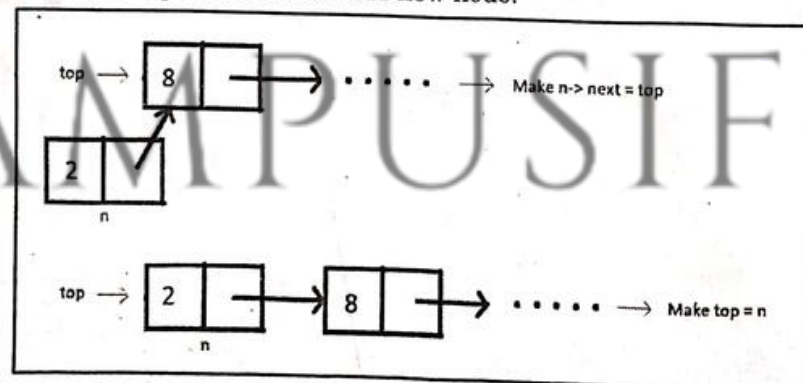


Figure 17.1 : Stack Using Linked List- (PUSH operation)

Algorithm :

PR-17

Initialize the stack :

- Start with an empty stack (top is initially set to NULL)

Push Operation :

- Input : A value data to be pushed onto the stack
- Step 1 : Create a new node using dynamic memory allocation
- Step 2 : Set the data of the new node to the input value
- Step 3 : Set the next pointer of the new node to point to the current top node.
- Step 4 : Update the top pointer to point to the new node.
- The element is now successfully pushed onto the stack

Pop Operation :

- Input : None
- Step 1 : Check if the stack is empty ($top == NULL$)
 - If true, print "Stack Underflow" and return an error value
- Step 2 : Save the data from the top node
- Step 3 : Update the top pointer to the next node in the stack
- Step 4 : Free the memory of the old top node.
- The element is now successfully popped from the stack and returned

Display Operation :

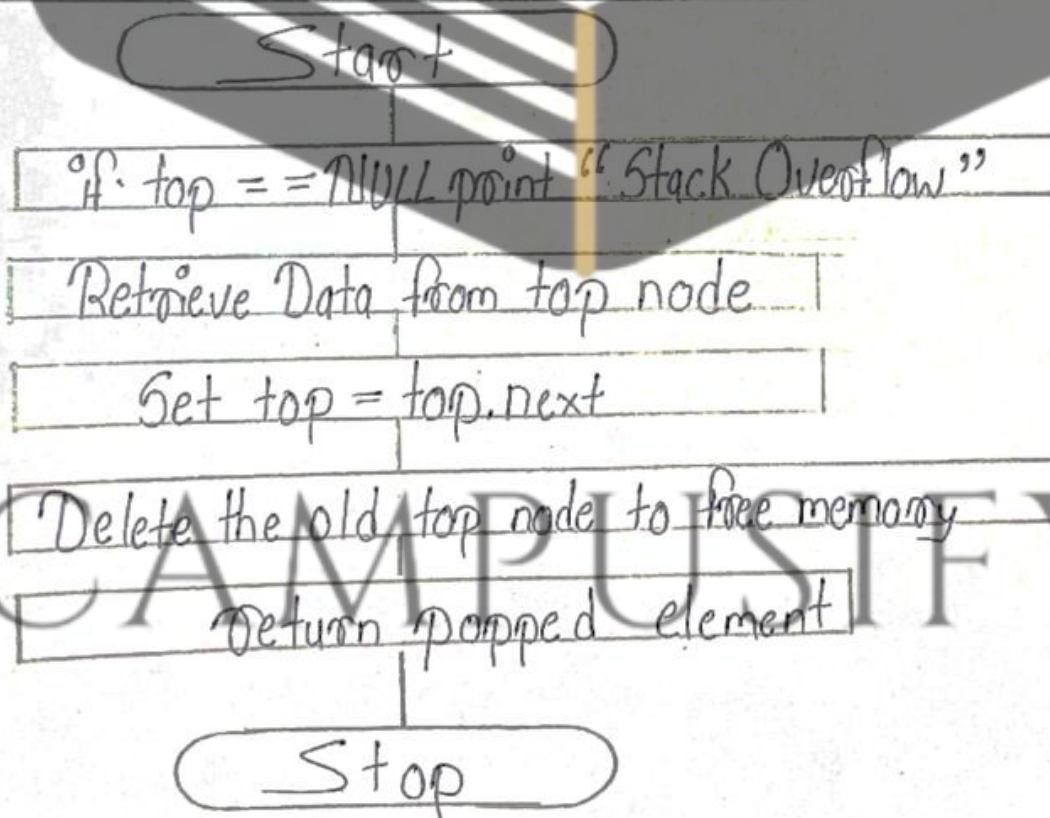
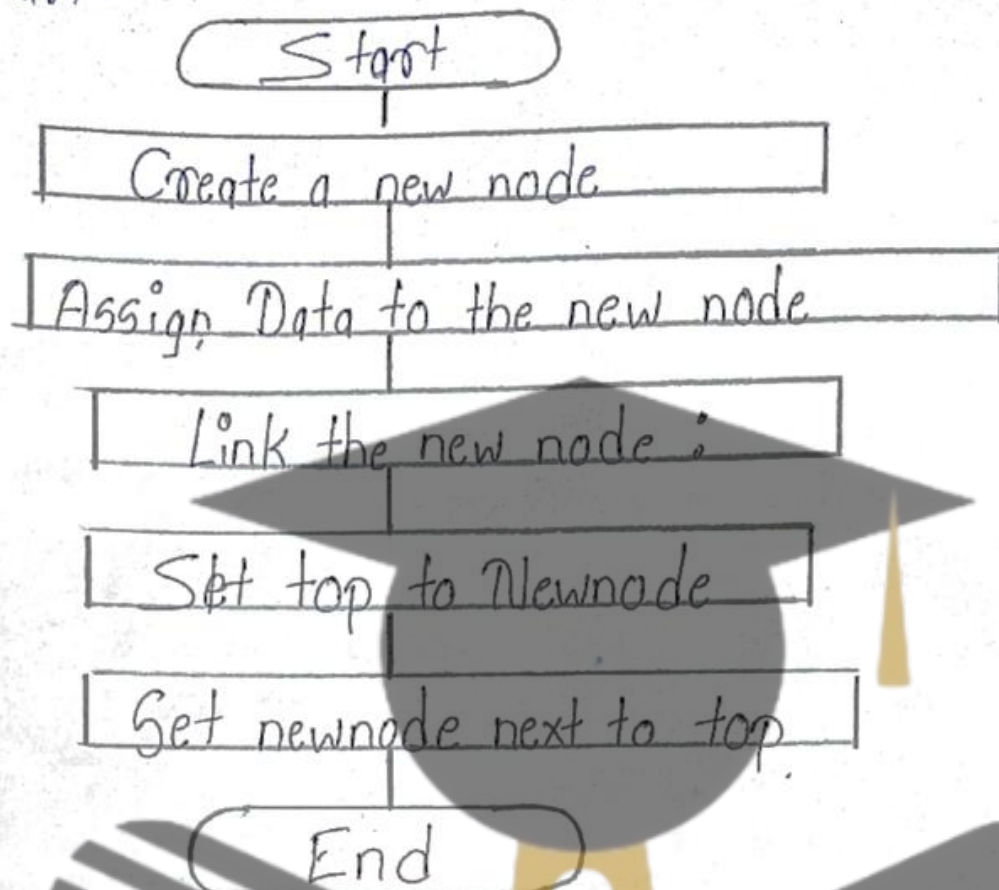
- Input : None
- Step 1 : Start from the top node
- Step 2 : Traverse the stack, printing the data of each node
- Step 3 : Continue ~~un~~ untill the end of the stack
(NULL is reached)

End :

- The program demonstrates the stack operations through a series of push and pop operations and displays the state of the stack after each operation.

CAMPUSIFY

Flowchart:



(IX) C Program Code

```
#include <stdio.h>
#include <conio.h>
```

```
struct Node {
    int data;
    struct Node * next;
}

void push (struct Node * *top, int data)
{
    struct Node * newNode = (struct Node *) malloc
    (sizeof (struct Node));
    if (!newNode) {
        printf ("Stack Overflow \n");
        return;
    }
    newNode->data = data;
    newNode->next = *top;
    *top = newNode;
    printf ("Pushed %d onto the stack \n", data);
}

int pop (struct Node * *top) {
    if (*top == NULL) {
        printf ("Stack Underflow \n");
        return -1;
    }
}
```

C Program Code : Continue

```
struct Node* temp = *top;
int poppedData = temp->data;
*top = (*top)->next;

free(temp);

return poppedData;
}

void display (struct Node *top)
{
    if (top == NULL) {
        printf("Stack is Empty\n");
        return;
    }
    printf("Stack elements : ");
    while (top != NULL) {
        printf("%d", top->data);
        top = top->next;
    }
    printf("\n");
}

int main()
{
    struct Node* stack = NULL;
    push(&stack, 10);
    push(&stack, 20);
}
```

```
push(&stack, 30)  
display(stack);  
printf("Popped %d from the stack \n",  
      pop(&stack));
```

```
display(stack);
```

```
printf("Popped %d from the stack \n",  
      pop(&stack));  
display(stack);  
return 0;
```

CAMPUSIFY



(X) Resources required

Sr. No.	Equipment Name with broad specification	Relevant LLO Number
1	Computer System with all necessary Peripherals and Internet Connectivity 'C' Compiler / GCC Compiler/ Online 'C' Compiler All	ALL

(XI) Precautions to be Followed

1. Ensure that all C statements must end with a semicolon (;).
2. Use white spaces in c to describe blanks and tabs.
3. Ensure use of Dynamic memory allocation to store Stack data
4. Follow safety practices.

(XII) Result (Output of Program)

Pushed 10 onto the stack
 Pushed 20 onto the stack
 Pushed 30 onto the stack
 Stack contents:

30
 20
 10

Popped 30 from the stack
 Popped 20 from the stack
 Stack contents:

10

(XIII) Conclusion

We successfully Completed Data Structure Using C (313301)
 Practised No.17 and Write a 'C' Program to perform PUSH
 and POP Operations on Stack using Linked List.

(XIV) Practical Related Questions

1. Write a C program to perform following operations on Stack as Linked List. PUSH (10), POP, PUSH (20), POP, PUSH (10), POP. (10)
2. What is the time complexity of the push, pop, and peek operations in your implementation?

(XV) Exercise

1. What are the advantages of using a linked list to implement a stack?
2. Give limitations of using Linked List based Stack.

[Space to Write for Answers]

PRQ: Q1) Program -

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
int isEmpty (struct Node* top)
```

```
{
```

```
    return top == NULL;
```

```
}
```

```
void push (struct Node** top, int value)
```

```
{
```

```
    struct Node* newNode = (struct Node*) malloc
```

```
    (sizeof (struct Node));
```

```
    if (!newNode) {
```

```
        printf ("Stack overflow, cannot push %d\n", value);
```

```
        return;
```

```
    }
```

```
    newNode->data = value;
```

```
    newNode->next = *top
```

```
    *top = newNode;
```

```
    printf ("Pushed %d onto the stack.\n", value);
```

```

int pop(struct Node **top) {
    if (isEmpty(*top)) {
        printf("Stack underflow, cannot pop.\n");
        return -1;
    }

```

```

    struct Node * temp = *top;
    int poppedValue = temp->data;
    *top = (*top) -> next;
    free(temp);
    printf("Popped %d from the stack.\n",
        poppedValue);
    return poppedValue;
}

```

(XVI) References / Suggestions for further Reading Software/Learning Websites

<https://www.scholarhat.com/tutorial/datastructures/binary-search-tree-in-data-structures>

(XVII) Assessment Scheme

	Performance Indicators	Weightage %
	Process related (30 Marks)	60%
1.	Debugging ability	20%
2.	Correctness of Program codes	30%
3.	Quality of output achieved (LLO mapped)	10%
	Product related : 20 Marks	40%
4.	Completion and submission of practical in time	20%
5.	Answer to sample questions	20%
	Total (50 Marks)	100%

List of Student / Team Members

(1) Prath Pande (2) Shivam Jannik
 (3) (4)

Marks Obtained		Dated signature of Teacher
Process Related (30)	Product Related (20)	
	Total (50)	

PRQ : Q1) Continue -

```
int main()
{
    struct Node* top = NULL;
    Push(&top, 10);
    Push(&top, 20);
    Pop(&top);
    Push(&top, 10);
    Push(&top, 20);
    Pop(&top);
    Push(&top, 20);
    Pop(&top);
    return 0;
}
```

PRQ : Q2) -

In the stack implementation using a linked list, the time complexities of the PUSH, POP, and PEEK operations are as follow :

1) PUSH Operation

- Operation : Inserts a new element at the top of the stack
- Time Complexity :
 - i) $O(1)$ (constant time)
 - ii) Explanation : A new node is created and linked at the head of the list, which involves updating the pointer to the new to element. This operation does not require traversing the stack, so it takes constant time.

2) POP Operation :

⊙ Operation : Removes the top element from the stack.

⊙ Time Complexity :

- $O(1)$ (Constant time)

- Explanation : The top element is removed, and the top pointer is updated to point to the next node. This also does not require traversal of the stack, so the operation takes constant time.

3) PEEK Operation :

⊙ Operation : Returns the top element of the stack without removing it.

⊙ Time Complexity :

- $O(1)$ (Constant time)

- Explanation : The top element is accessed directly using the pointer to the top of the stack, and no traversal is required, so it takes constant time.

Summary :

- PUSH : $O(1)$
- POP : $O(1)$
- PEEK : $O(1)$

All three operations run in constant time because they only involve updating pointers and accessing the top element of the stack without any need to traverse the stack. This makes stack operations efficient when using a linked list.

✓ Practical No. 18 : Write a 'C' Program to Perform Multiplication of Two Numbers using Recursion

(I) Practical Significance

In essence, recursion is a fundamental concept in computer science and mathematics that provides a method for solving problems by breaking them down into smaller, more manageable sub-problems. Particularly in Data Structure, it is a powerful tool to work with recursive data structures like trees and graphs. For example, tree traversals (in-order, pre-order, post-order) are most easily implemented using recursion. Similarly, many graph algorithms, such as depth-first search (DFS), are naturally expressed recursively. Some classical algorithms like Merge Sort, Quick Sort use a recursive divide-and-conquer strategy. Hence by storing the results of subproblems (memorization), recursive algorithms can be optimized to avoid redundant computations.

(II) Industry/ Employer Expected Outcome

Through this practical student will,

1. Understand the concept of recursion.
2. Write an algorithm to perform given task using Recursion.
3. Implement C program for the designed algorithm using concept of Recursion.
4. Save/Compile/ Debug/ the C program.
5. Check for the desired output.

(III) Course Level Learning Outcomes

Demonstrate multiplication of two numbers using recursion.

(IV) Laboratory Learning Outcome

Perform Operations on the Stack using a Linked List.

(V) Relevant Affective domain related Outcome(s)

1. Follow safety practices.
2. Follow ethical practices.

(VI) Relevant Theoretical Background

Recursion

In programming, a recursive function is a function that calls itself in order to break down the problem into more manageable sub-problems.

Example

Multiplication using Recursion

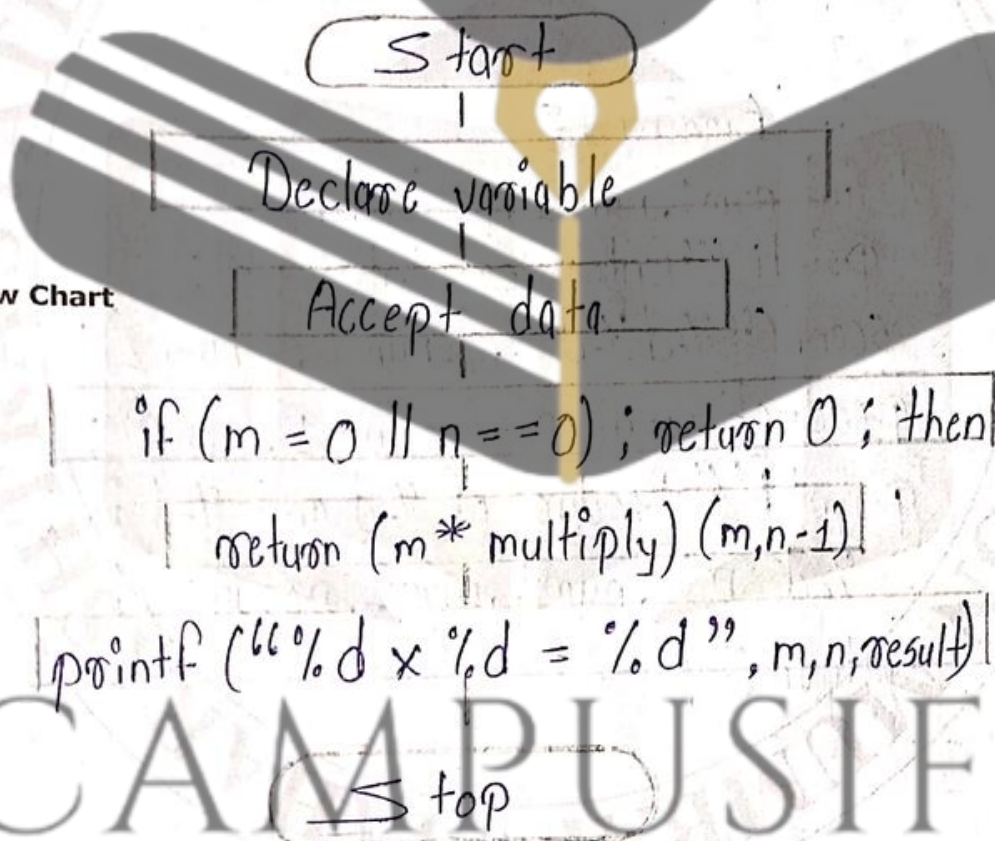
In high-level languages, there is the multiplication operators available to directly perform multiplication. However, the multiplication is actually a repetitive addition. So the result of $A * B$ is repetitive addition of A, B number of times, or we can say repetitive addition of B, A number of times. Whenever there is a repetition, we can do this using recursion.

Logic

1. Get the inputs from the user and store it in the variables x and y,
2. The function product is used to calculate the product of given numbers,
3. A nested if-else statement is used to check x is less than y,
4. if the condition is satisfied, add the value of x with the value, else execute the else-if condition,
5. check the value of y is not equal to 0,
6. If the condition statement is true, execute the statement, or if the condition is not satisfied execute else block,
7. Print the product of two numbers.

(VII) Algorithm :

1. Start
2. Take or Accept data
3. Multiply no.
4. Store result in result variable
5. print result
6. End

(VIII) Flow Chart

(IX) C Program Code : #include <stdio.h>
int multiply (int a, int b)

```
{  
    if (b == 0) {  
        return 0;  
    }  
    else if (b > 0) {  
        return a + multiply(a, b - 1);  
    }  
    else {  
        return -multiply(a, -b);  
    }  
}
```

```
{  
    int &main() {  
        int num1, num2, product;  
        printf("Enter two numbers : ");  
        scanf("%d %d", &num1, &num2);  
        product = multiply(num1, num2);  
        printf("Product of %d and %d is :  
        %d \n", num1, num2, product);  
        return 0;  
    }  
}
```

(X) Resources required

Sr. No.	Equipment Name with broad specification	Relevant LLO Number
1	Computer System with all necessary Peripherals and Internet Connectivity 'C' Compiler / GCC Compiler/ Online 'C' Compiler All	ALL

(XI) Precautions to be Followed

1. Ensure that all C statements must end with a semicolon(;).
2. Use white spaces in c to describe blanks and tabs.
3. Follow safety practices.

(XII) Result (Output of Program)

Enter two numbers : 2 3
Product of 2 and 3 is : 6

(XIII) Conclusion

Each of the three recursive C programs effectively ~~don~~ demonstrates the use of recursion, utilizing the call stack to manage function calls and correctly solve problems such as calculating factorials, summing numbers, and generating Fibonacci series.

(XIV) Practical Related Questions

1. Implement a C program to print Fibonacci series using recursion.
2. Write a program in C to calculate the sum of numbers from 1 to n using recursion.

(XV) Exercise

1. Define a term recursion and give its advantages.
2. Describe how stack is used in recursion with example.

[Space to Write for Answers]

PRQ: Q1) Program -

```
#include <stdio.h>
```

```
int fibonacci(int n) {
```

```
    if (n == 0) {
```

```
        return 0;
```

```
    } else if (n == 1) {
```

```
        return 1;
```

```
    } else {
```

```
        return fibonacci(n-1) + fibonacci(n-2);
```

```
    }
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter the number of terms in the Fibonacci series: ");
```

```
    scanf("%d", &n);
```

```
    printf("Fibonacci series: ");
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        printf("%d ", fibonacci(i));
```

```
    }
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

PRQ : Q2) Program -

```
#include <stdio.h>
```

```
int sumOfNumbers (int n)
```

```
{  
    if (n == 1)
```

```
{  
        return 1;
```

```
} else {
```

```
    return n + sumOfNumbers (n - 1);
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
    int n;
```

```
    printf ("Enter a number : ");
```

```
    scanf ("%d", &n);
```

```
    int sum = sumOfNumbers (n);
```

```
    printf ("Sum of Numbers from 1 to %d is : %d\n",  
           n, sum);
```

```
    return 0;
```

```
}
```

CAMPUSIFY

✓ Practical No. 21 : Write a 'C' Program to Perform INSERT and DELETE Operations on Linear Queue using an Array

(I) Practical Significance

The ability to manage ordered sequences of elements and handling of tasks and resources efficiently makes the use of queue indispensable in various applications across different fields, from computing and networking to business operations and service management. It is a linear data structure works in First in First (FIFO) Order with two operational ends Front End to add new element and Rear End to delete or process new element.

(II) Industry/ Employer Expected Outcome

Through this practical student should,

1. Understand basic operations to be performed on Linear Queue.
2. Write an algorithm to perform basic operations on Linear Queue.
3. Implement C program to Queue as ADT using Array.
4. Save/Compile/ Debug/ the C program.
5. Check for the desired output.

(III) Course Level Learning Outcomes

Implement basic operations on linear queue using Array.

(IV) Laboratory Learning Outcome

Perform Operations on Linear Queue using Array.

(V) Relevant Affective domain related Outcome(s)

1. Follow safety practices.
2. Follow ethical practices.

(VI) Relevant Theoretical Background Linear Queue using Array

Queue data structure is a linear data structure in which the operations are performed based on FIFO principle.

Queue is an object (an abstract data structure - ADT) that allows the following operations: Enqueue: Add an element to the rear end of the queue.

Dequeue: Remove an element from the front of the queue. IsEmpty: Check if the queue is empty.

IsFull: Check if the queue is full.

Peek: Get the value of the front of the queue without removing it.

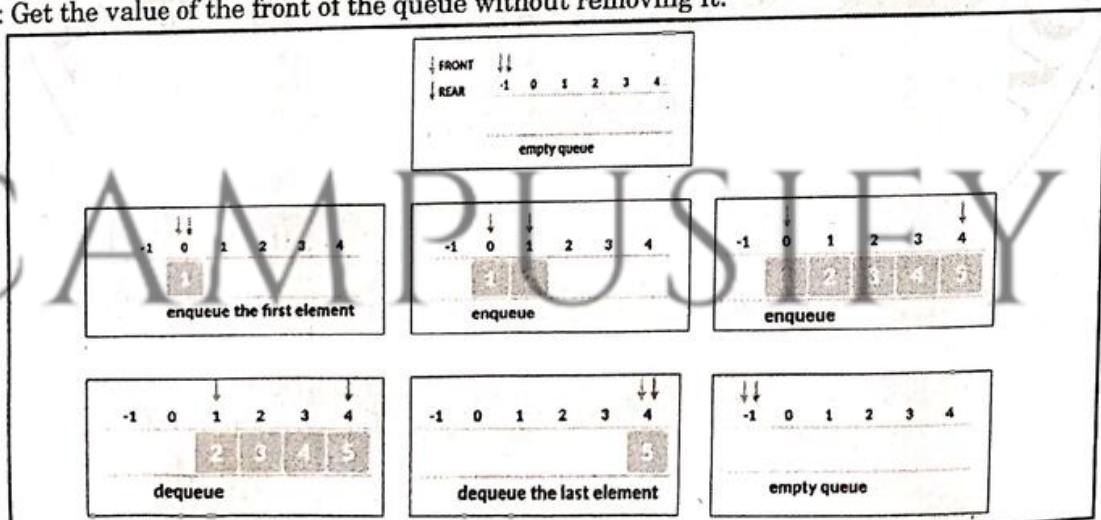


Figure 21.1 : Insertion (enqueue) and Deletion (dequeue) operations on Linear Queue

(VII) Algorithm

Initialization:

Initialize front and rear to -1 to indicate an empty queue.

Enqueue:

Check if the queue is full (rear has reached the maximum size)

If not full:

Increment rear.

Insert the new element at queue [rear]

If the queue was initially empty (front was -1), set front to 0.

Dequeue:

Check if the queue is empty (front is -1)

~~————~~ If not empty:

If there's only one element in the queue ($\text{front} == \text{rear}$), set both front and rear to -1.

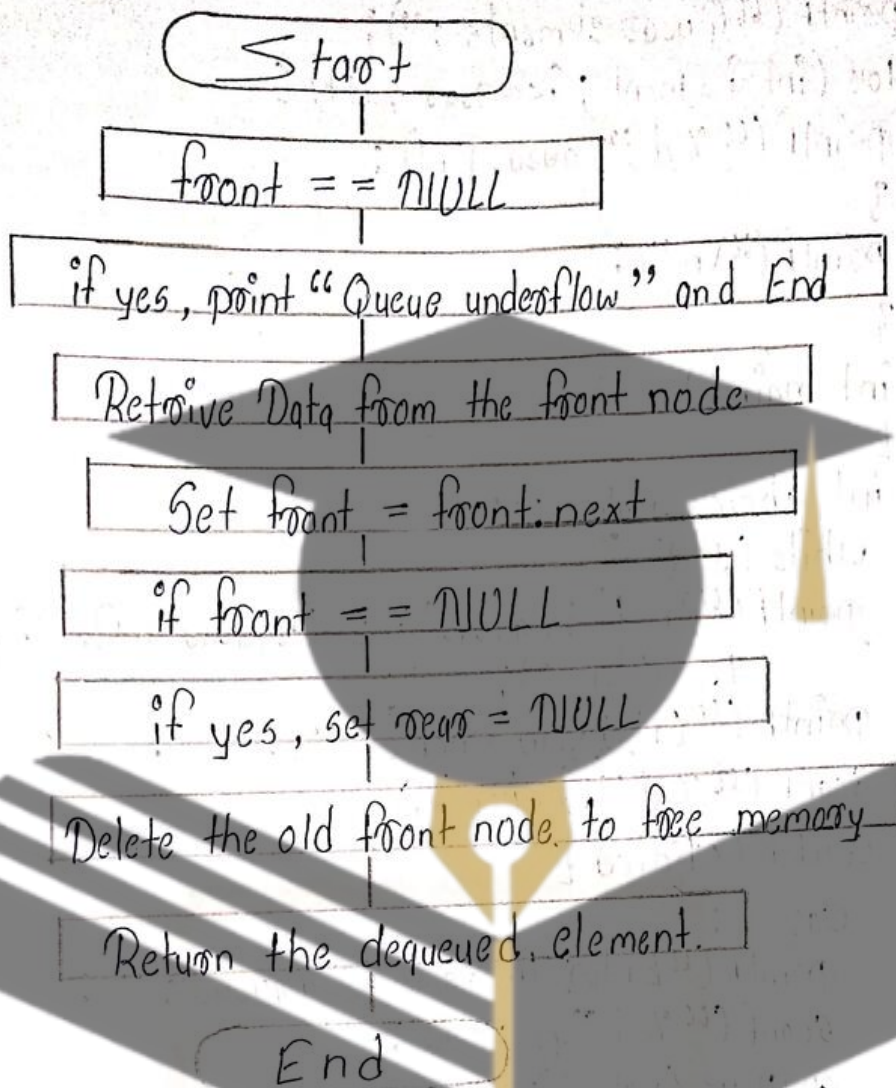
Otherwise, increment front to access the next element

Display:

Check if the queue is empty

If not empty, iterate from front to rear and print each element in the queue.

Flowchart:



CAMPUSIFY

(IX) C Program Code

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 5

int queue [MAX_SIZE];
int front = -1;
int rear = -1;

void enqueue (int value)
{
    if (rear == MAX_SIZE - 1) {
        printf ("Queue is full \n");
        return;
    }
    if (front == -1) {
        front = 0;
    }
    rear++;
    queue [rear] = value;
    printf ("%d enqueued to the queue \n", value);
}

void dequeue ()
{
    if (front == -1) {
        printf ("Queue is empty \n");
        return;
    }
}
```

PR-21
C Program Code : Continue -

```
printf("Queue elements :");  
for (int i = front ; i <= rear ; i++) {  
    printf("%d ", queue[i]);  
}  
printf("\n");  
}  
int main()  
{  
    int choice, value;  
    while (1) {  
        printf("\n 1. Enqueue\n 2. Dequeue\n 3. Display\n 4. Exit\n");  
        printf("Enter your choice :");  
        scanf("%d", &choice);  
        switch (choice) {  
            Case 1 :  
                printf("Enter the value to enqueue :");  
                scanf("%d", &value);  
                enqueue(value);  
                break;  
            Case 2 :  
                dequeue();  
                break;  
            Case 3 :  
                display();  
                break;  
            Case 4 :  
                exit(0);  
            default :  
                printf("Invalid choice\n");  
        }  
    }  
    return 0;  
}
```

(X) Resources required

Sr. No.	Equipment Name with broad specification	Relevant Number	LLO
1	Computer System with all necessary Peripherals and Internet Connectivity 'C' Compiler / GCC Compiler/ Online 'C' Compiler All	ALL	

(XI) Precautions to be Followed

1. Ensure that all C statements must end with a semicolon (;).
2. Use white spaces in c to describe blanks and tabs.
3. Ensure use of recursive function call
4. Follow safety practices.

(XII) Result (Output of Program)

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter the value to enqueue: 235

235 enqueued to the queue

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 3

Queue elements: 235

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice:

(XIII) Conclusion

C code effectively implements a linear queue using an array, demonstrating proper enqueue, dequeue, and display operations while handling queue overflow and underflow conditions.

(XIV) Practical Related Questions

1. Write a C program to implement a linear queue with operations for enqueue (15), enqueue (48), enqueue (69), dequeue, enqueue (12), enqueue (23), dequeue, dequeue checking if the queue is empty or full, and displaying the current queue.
2. Write a C program to demonstrate Queue Underflow and Queue Overflow.

(XV) Exercise

1. Describe Queue as Abstract Data Type (ADT).
2. Give applications of Queue in different domains.

[Space to Write for Answers]

PRO: Q1) Program -

```

#include <stdio.h>
#include <stdlib.h>
#define SIZE 5

int queue[SIZE];
int front = -1, rear = -1;

int isFull() {
    return rear == SIZE - 1;
}

int isEmpty() {
    return front == -1 || front > rear;
}

void enqueue(int value) {
    if (isFull()) {
        printf("Queue is full. Cannot enqueue %d.\n", value);
    } else {
        if (front == -1)
            front = 0;
        queue[++rear] = value;
        printf("Enqueued : %d\n", value);
    }
}

void dequeue() {
    if (isEmpty()) {

```

```

printf("Queue is empty. Cannot dequeue.\n");
} else {
printf("Dequeued : %d\n", queue[front]);
front++;
if (front > rear) {
front = rear = -1;
}
}
}

void displayQueue() {
if (isEmpty()) {
printf("Queue is empty.\n");
}
}

```

(XVI) References / Suggestions for further Reading Software/Learning Websites

1. <https://www.programiz.com/dsa/queue>
2. <https://ds1-iiith.vlabs.ac.in/exp/stacks-queues/queues/index.html>
3. <https://ds1-iiith.vlabs.ac.in/exp/stacks-queues/queues/queuesarrays.html>

(XVII) Assessment Scheme

	Performance Indicators	Weightage %
	Process related (30 Marks)	60%
1.	Debugging ability	20%
2.	Correctness of Program codes	30%
3.	Quality of output achieved (LLO mapped)	10%
	Product related : 20 Marks	40%
4.	Completion and submission of practical in time	20%
5.	Answer to sample questions	20%
	Total (50 Marks)	100%

List of Student /Team Members

- (1) Parth Pande (2) Shivam Jannik
 (3) (4)

Marks Obtained			Dated signature of Teacher
Process Related (30)	Product Related (20)	Total (50)	

Q1) Continue:

```

    } else {
        printf("Current Queue: ");
        for (int i = front; i <= rear; i++)
        {
            printf("%d ", queue[i]);
        }
    }
}

```

```

int main()

```

```

{
    enqueue(15);
    enqueue(48);
    enqueue(69);
    dequeue();
    enqueue(12);
    enqueue(23);
    dequeue();
    dequeue();

    displayQueue();
    return 0;
}

```

PRQ : Q2) Program:

#include <stdio.h>

#include <stdlib.h>

#define SIZE 5

int queue[SIZE];

int front = -1, rear = -1;

int isFull()

```
{  
    return rear == size - 1;  
}
```

```
void enqueue (int value) {  
    if (isFull()) {  
        printf("Queue Overflow ! Cannot enqueue %d.\n",  
            value);
```

```
    } else {  
        if (front == -1)  
            front = 0;
```

```
        queue[++rear] = value;  
        printf("Enqueued : %d\n", value);  
    }
```

```
}  
void dequeue () {  
    if (isEmpty()) {  
        printf("Queue Underflow ! Cannot dequeue.\n");
```

```
    } else {  
        printf("Dequeued : %d\n", queue[front]);  
        front++;  
        if (front > rear) {  
            front = rear = -1;  
        }
```

```
}
```

```
}
```

Q2) Continue:

PR-21(2)

```
void displayQueue ()
{
    if (isEmpty ())
    {
        printf ("Queue is empty.\n");
    } else {
        printf ("Current Queue : ");
        for (int i = front; i <= rear; i++)
        {
            printf ("%d ", queue[i]);
        }
        printf ("\n");
    }
}

int main ()
{
    printf ("Demonstrating Queue Underflow:\n");
    dequeue();
    printf ("\n Enqueuing elements:\n");
    enqueue(10);
    enqueue(20);
    enqueue(30);
    enqueue(40);
    enqueue(50);
    printf ("\n Demonstrating Queue Overflow:\n");
    enqueue(60);
    printf ("\n Displaying the queue:\n");
    displayQueue();
    return 0;
}
```